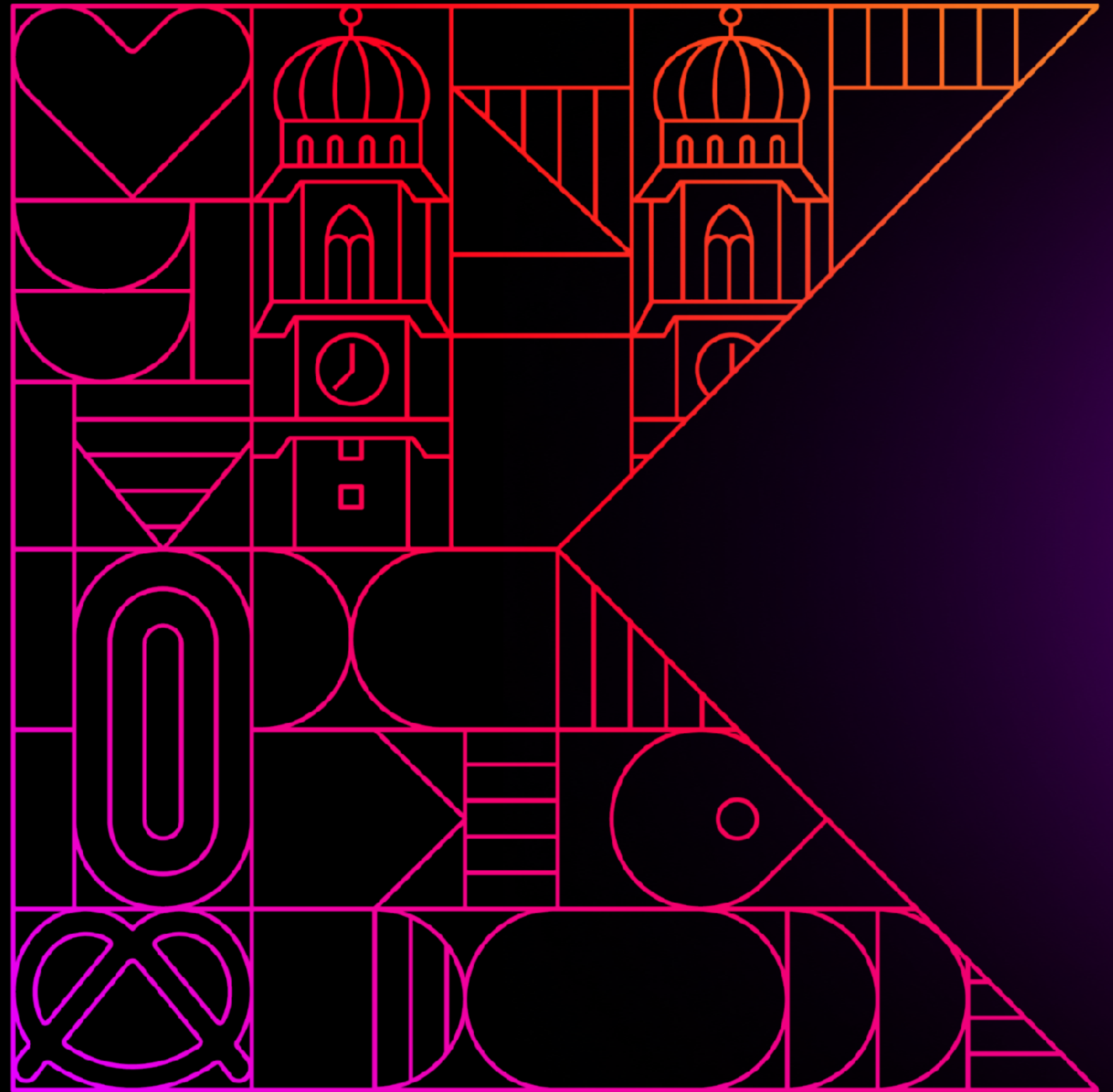


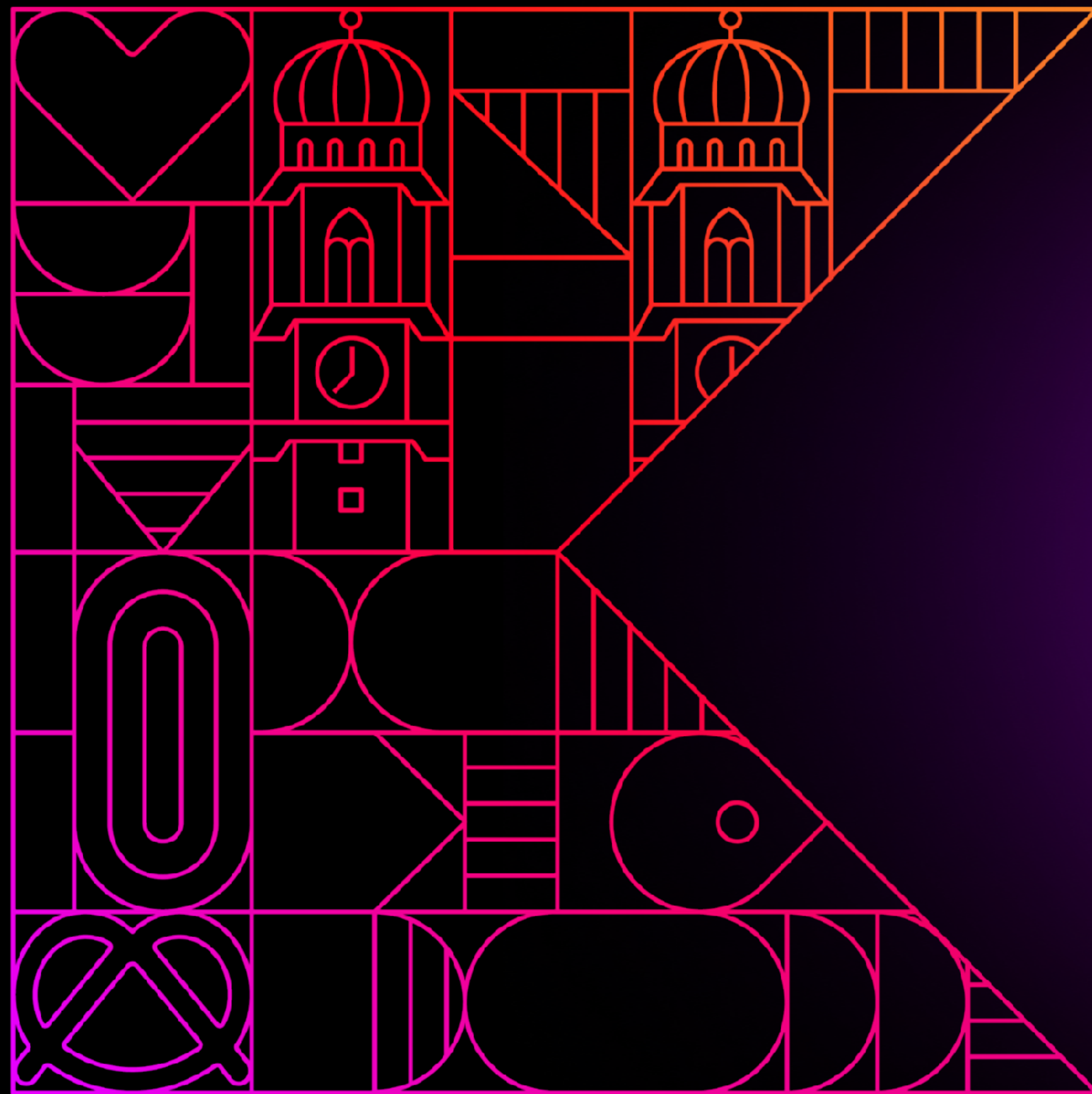
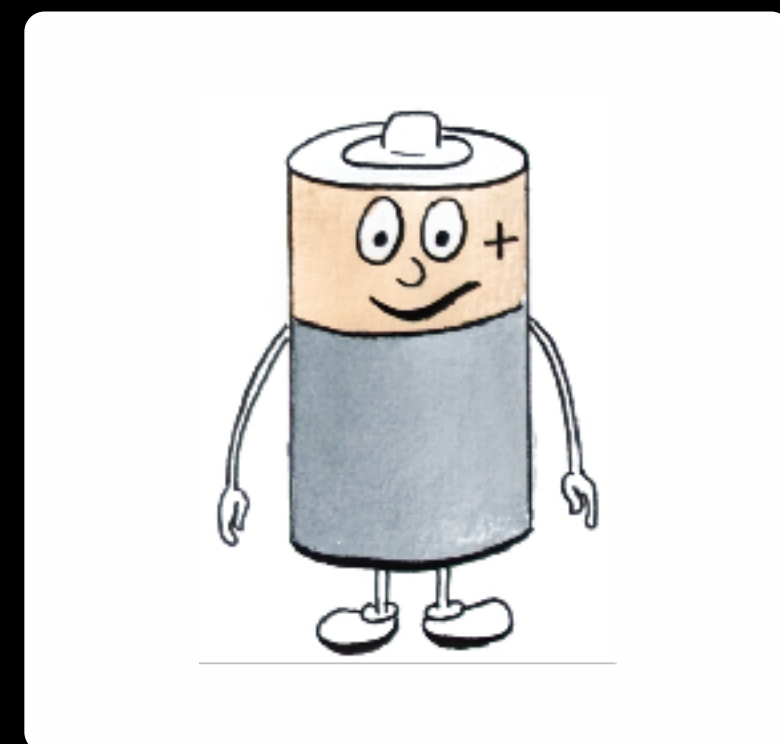
Redefining Machine Learning with Kotlin

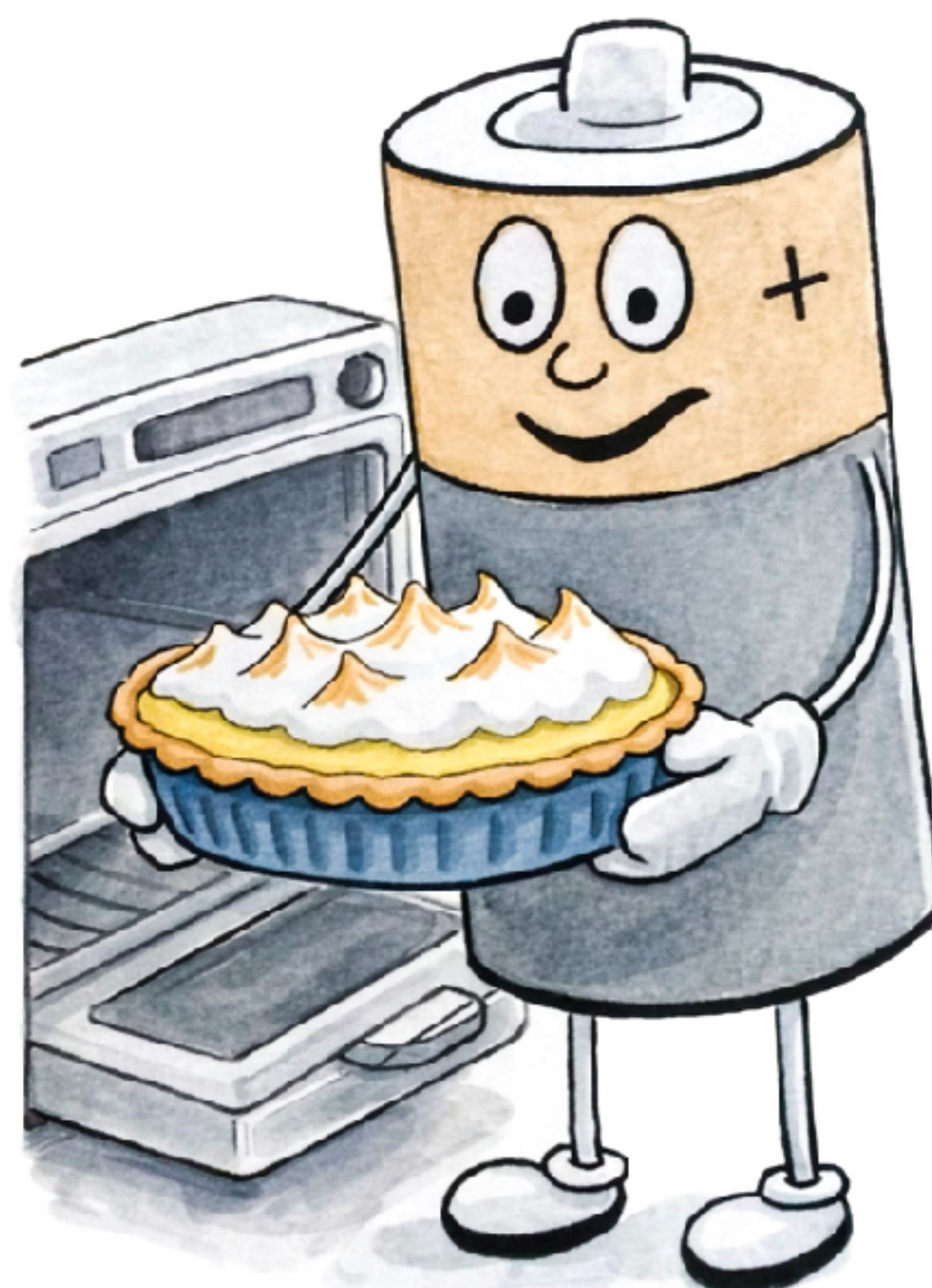
Michal Harakal
Deutsche Telekom AG



Redefining Machine Learning with Kotlin

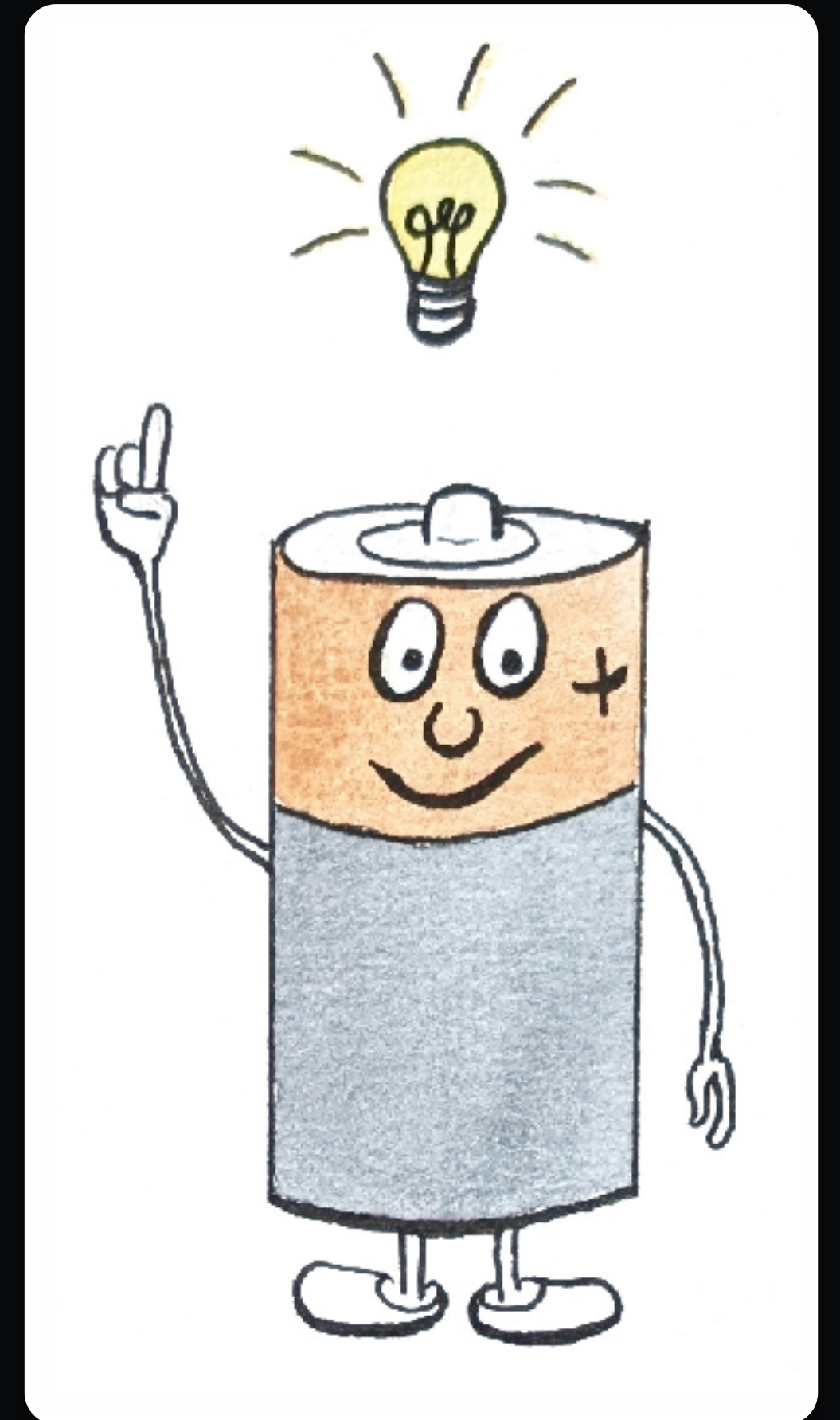
Michal Harakal
Deutsche Telekom AG





What I cannot create,
I do not understand.
Know how to solve every problem that
has been solved.

Richard Feynman, 1988



(C) Adriana Harakalova, 2024

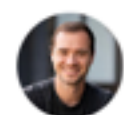
LET'S BUILD GPT. FROM SCRATCH. IN CODE. SPELLED OUT.



0:00 / 1:56:19 • intro: ChatGPT, Transformers, nanoGPT, Shakespeare >



Let's build GPT: from scratch, in code, spelled out.



Andrej Karpathy
540.000 Abonnenten

Abonnieren

👍 107.486



🔗 Teilen

🔖 Speichern



<https://www.youtube.com/watch?v=kCc8FmEb1nY>

Let's build GPT: from scratch, in code, spelled out.

Andrej Karpathy

- GTP-2 level model with about 10 million parameters
- 250 lines of code

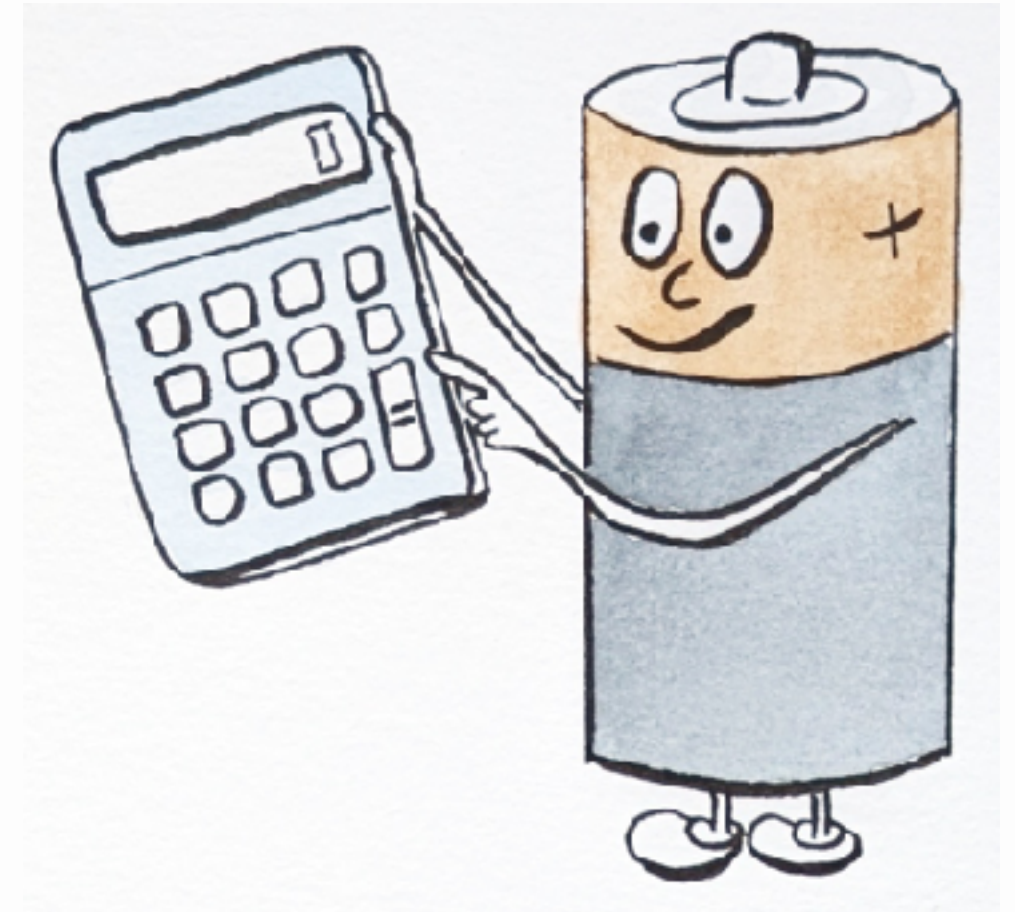
Let's build GPT: from scratch, in code, spelled out.

Andrej Karpathy

- GTP-2 level model with about 10 million parameters
- 250 lines of code
- written in PyTorch



Transformers 101



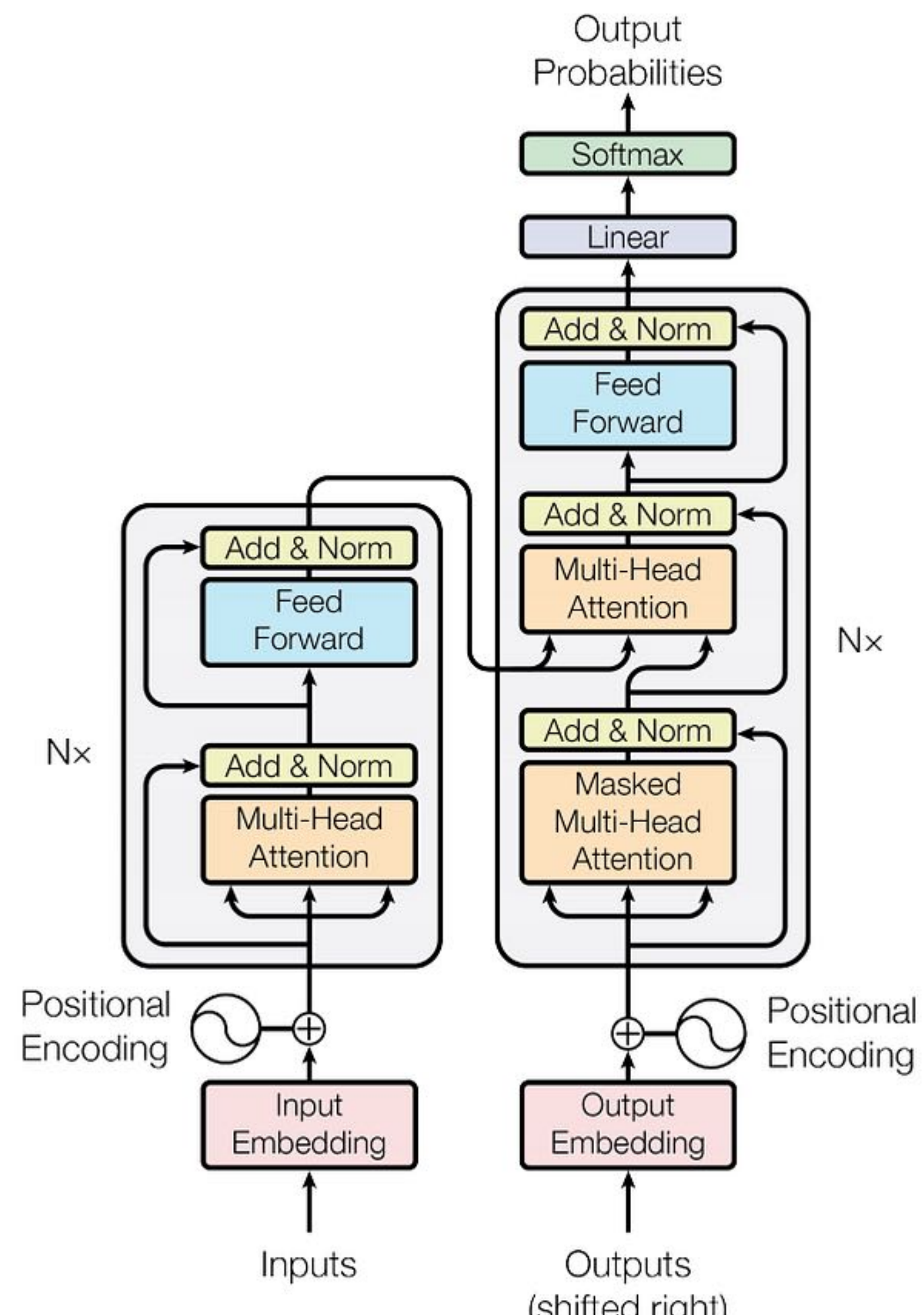
(C) Adriana Harakalova, 2024

Attention Is All You Need

Google Brain, Google Research et al.

- Published 2017
- Transformer model architecture
- Efficient computation as goal
- Encoder/Decoder for Translations

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin



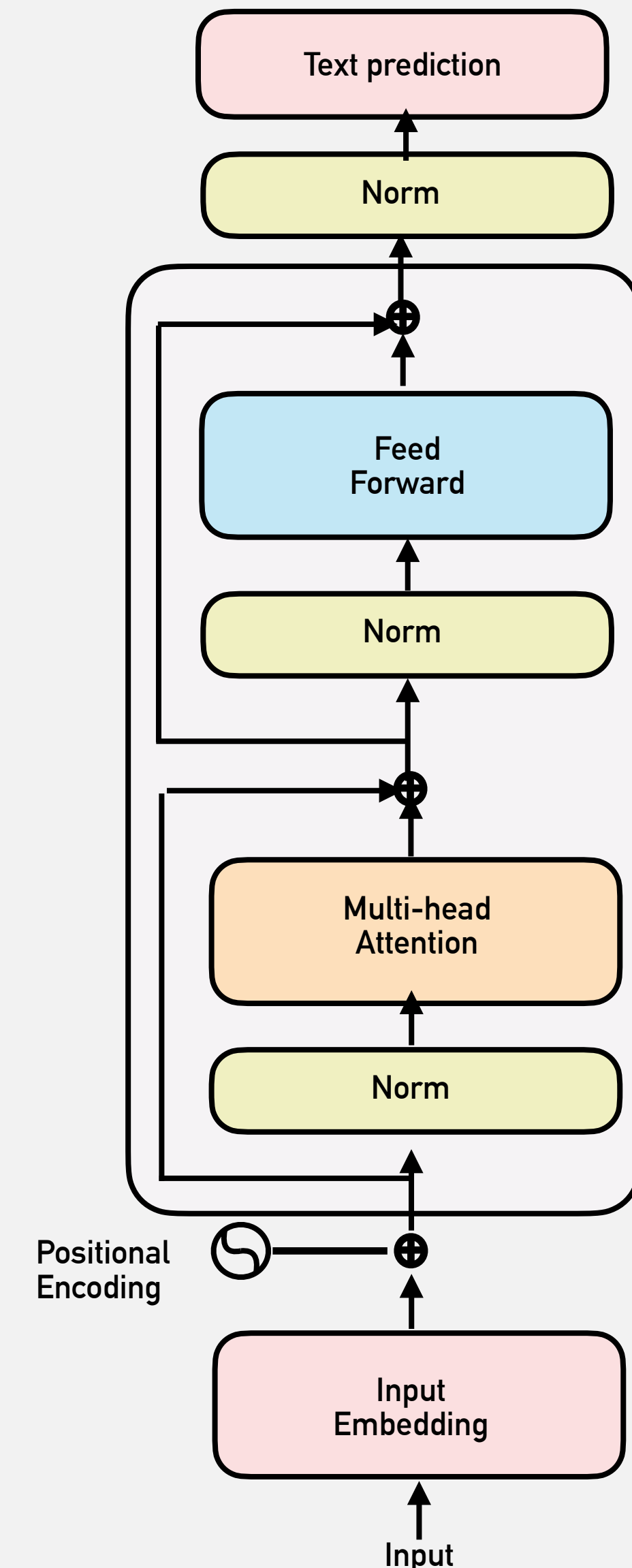
Decoder only

GTP-2

- 117M parameters
- 12 decoder layers
- 768 embedding

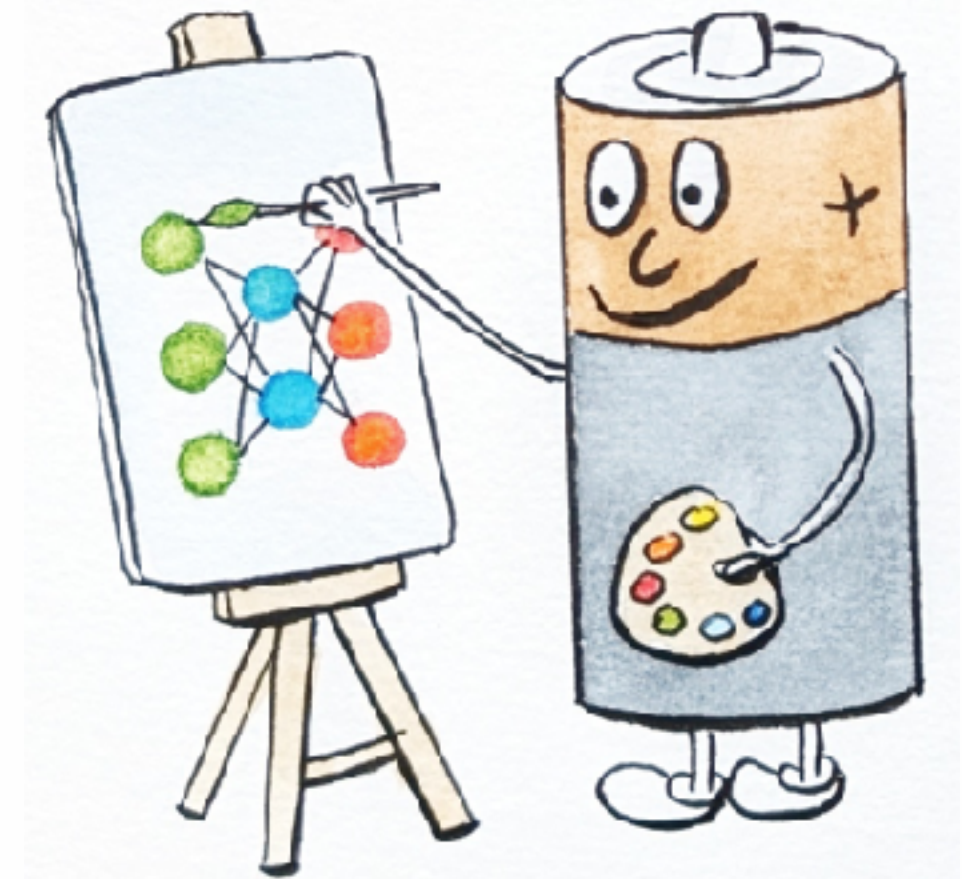
A. Karpathy's model

- 10M parameters
- 12 layers, 12 heads
- 1024 context



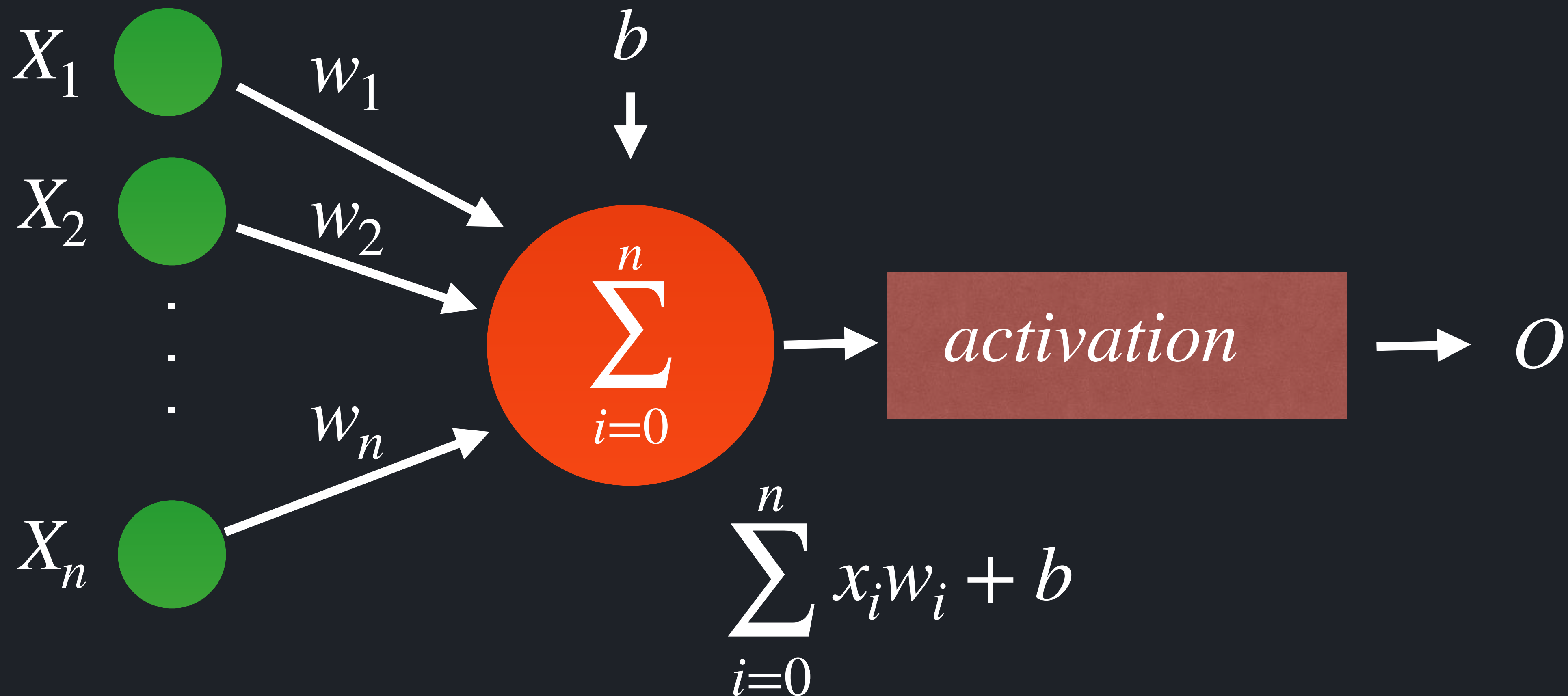
Deep neural networks

Feed
Forward

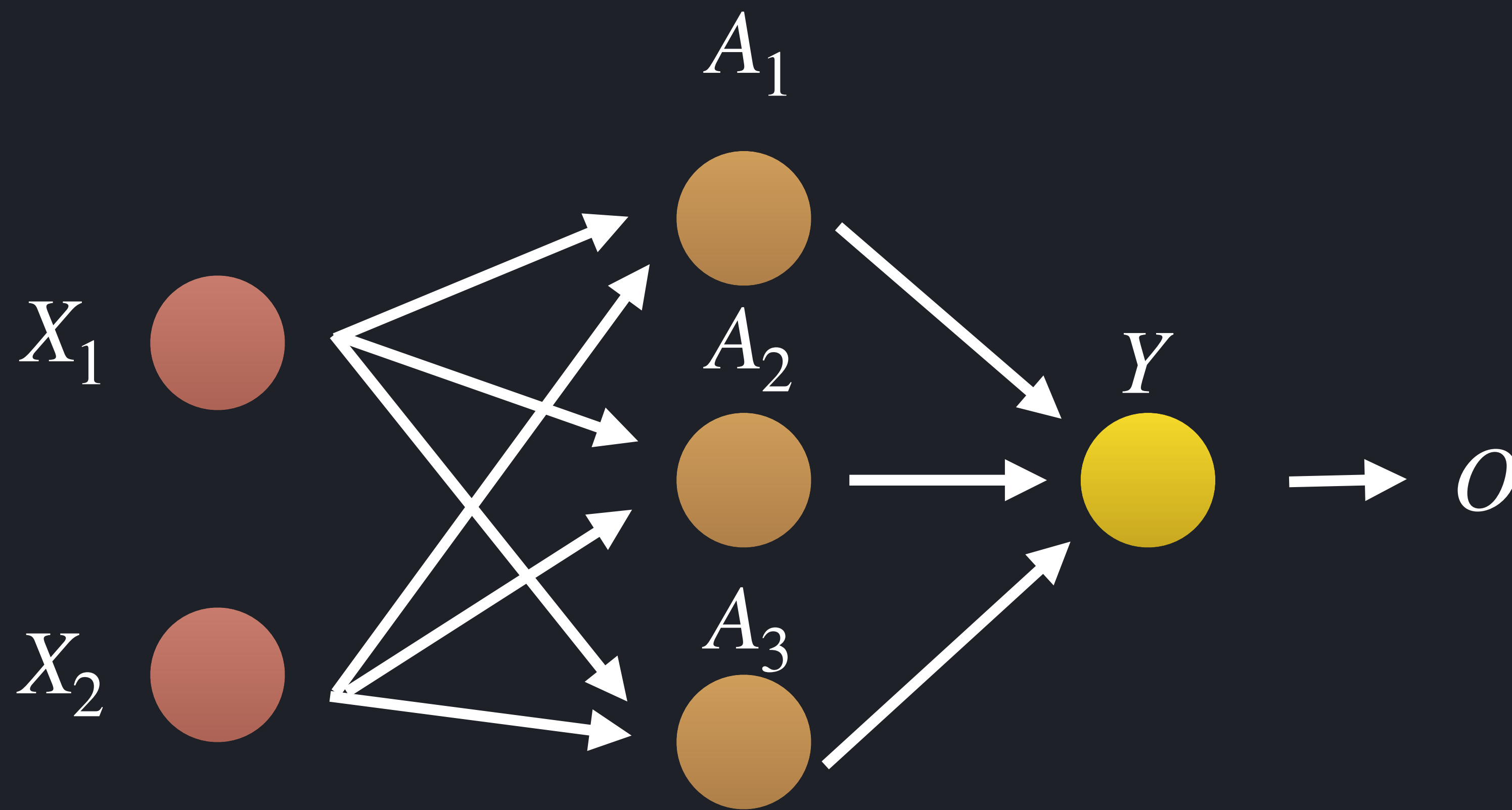


(C) Adriana Harakalova, 2024

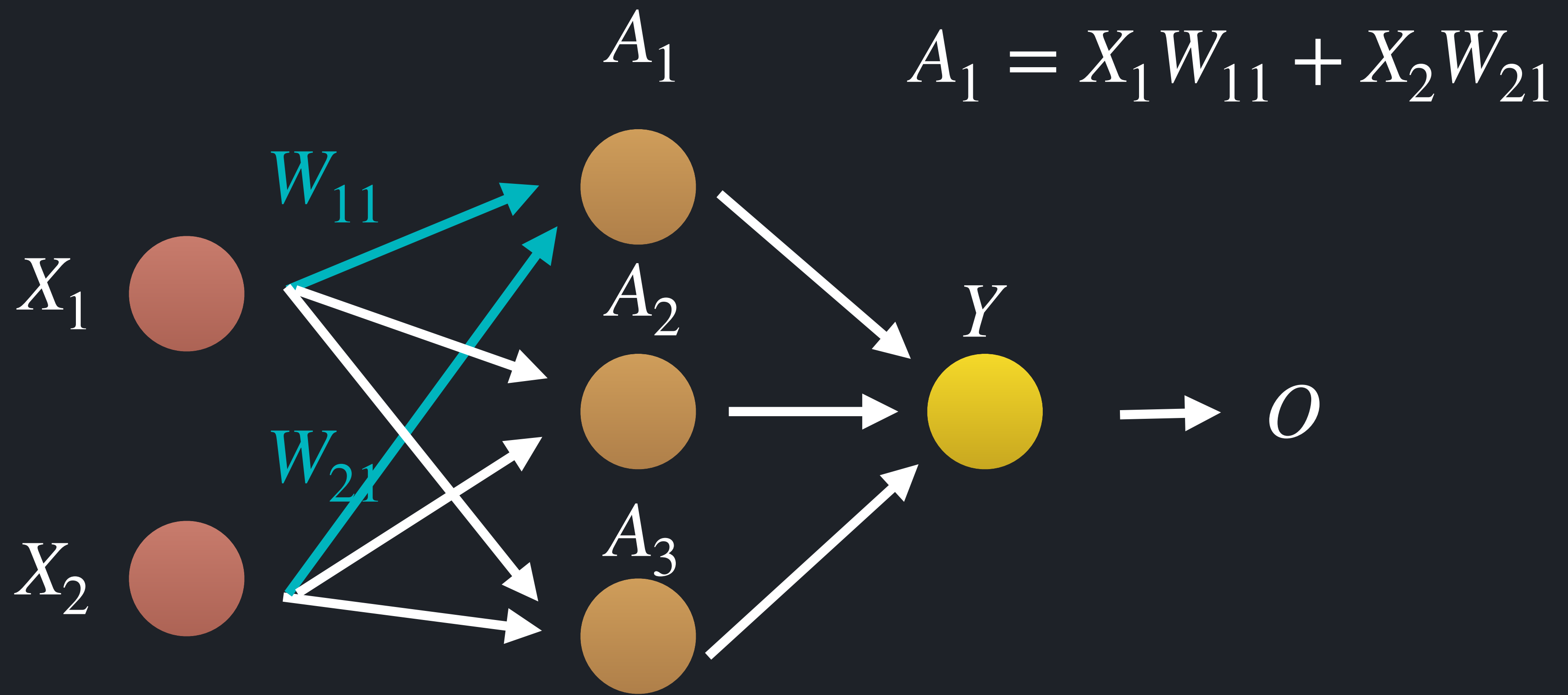
Neuron



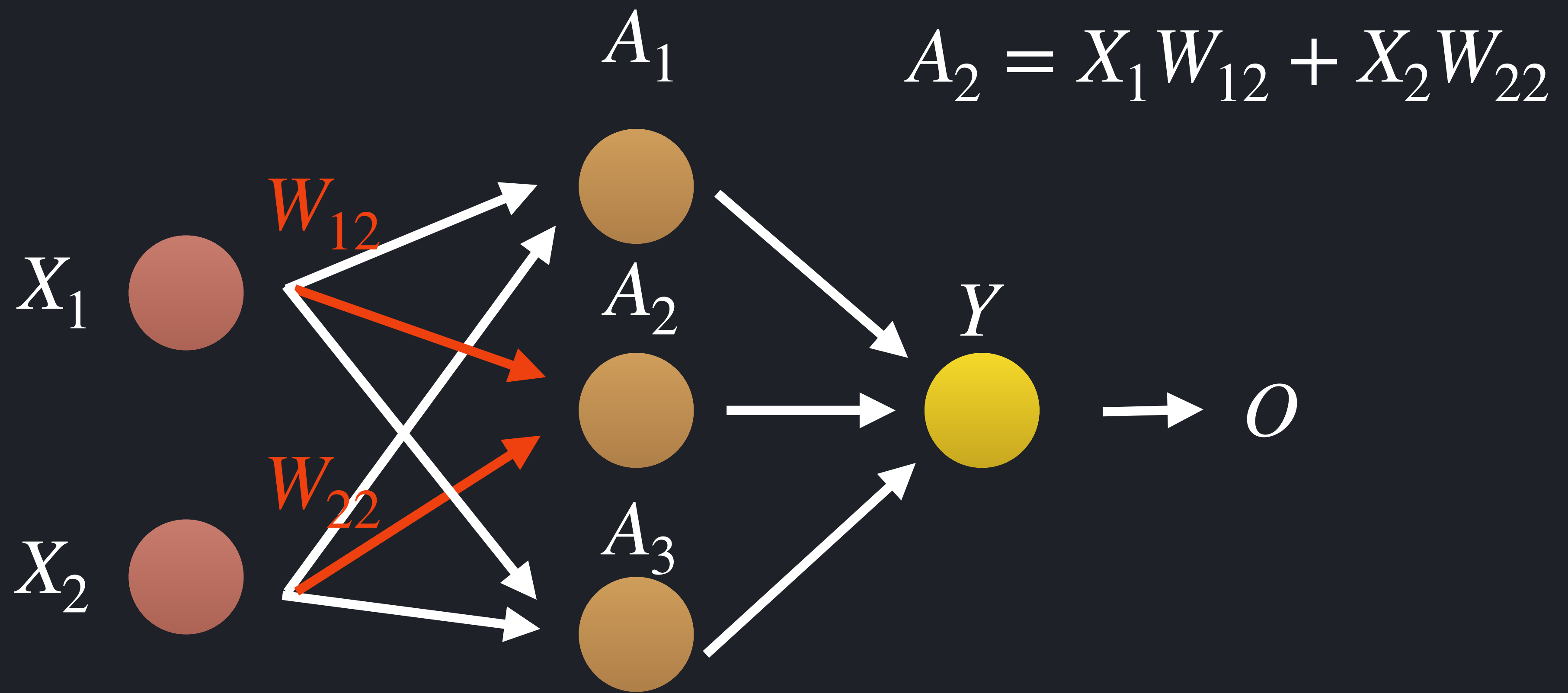
MLP - Multilayer Perceptron



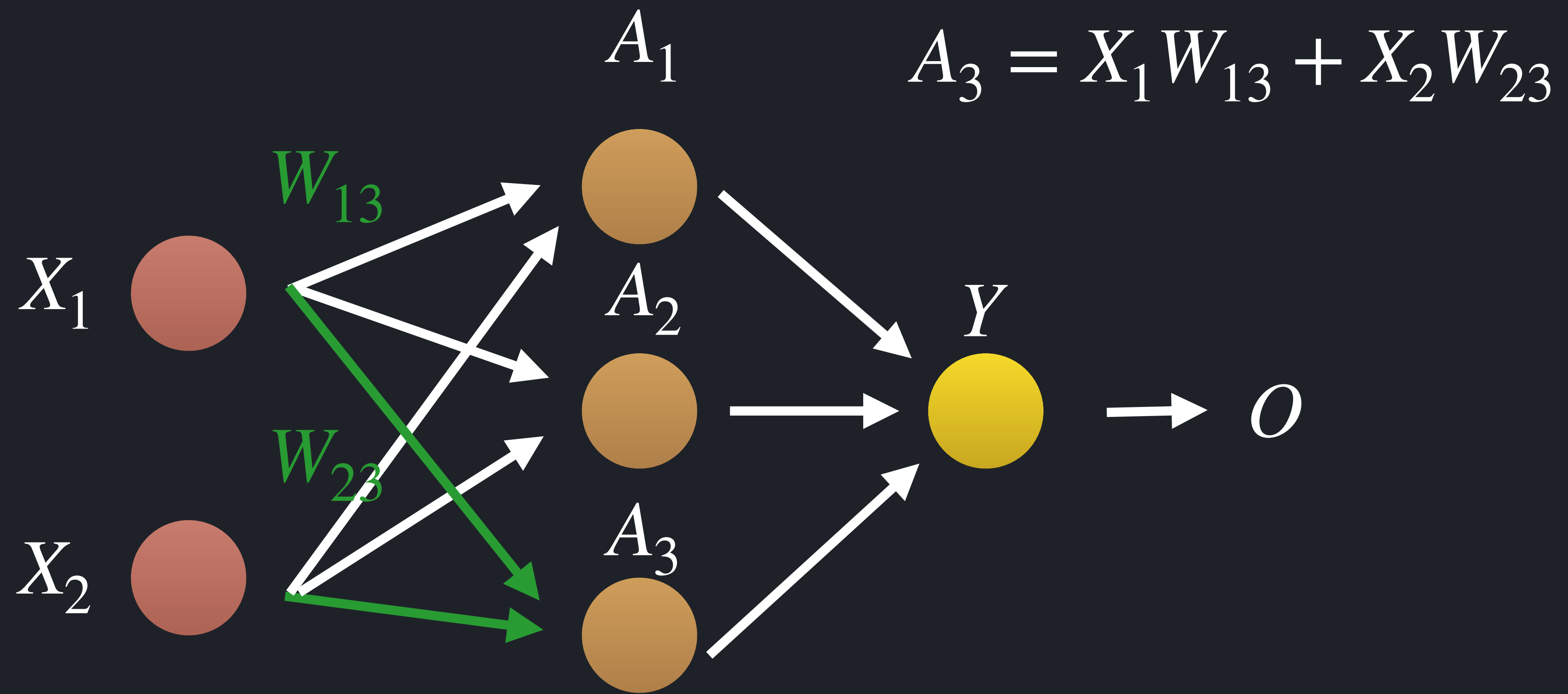
Forward propagation



Forward propagation



Forward propagation



Matrix Multiplication

$$A_{11} = X_1 W_{11} + X_2 W_{21}$$

$$A_{12} = X_1 W_{12} + X_2 W_{22}$$

$$A_{13} = X_1 W_{13} + X_2 W_{23}$$

$$A = X \cdot W$$

$$W = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix}$$

$$X = \begin{bmatrix} X_1 & X_2 \end{bmatrix}$$

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} \end{bmatrix}$$

$$A = X \cdot W$$

Matrices multiplication

$$A = X \cdot W$$

In Kotlin with lists of doubles

```
class Matrix(private val data: List<List<Double>>) {  
    private val numRows = data.size  
    private val numCols = data.firstOrNull()?.size ?: 0  
  
    fun matmul(other: Matrix): Matrix {  
        val resultData = List(numRows) { i ->  
            List(other.numCols) { j ->  
                (0 until numCols).sumOf { k ->  
                    data[i][k] * other.data[k][j]  
                }  
            }  
        }  
        return Matrix(resultData)  
    }  
}
```

Faster matrices multiplication

$$A = X \cdot W$$

Support for SIMD - DoubleArray

```
class Matrix(  
    val rows: Int,  
    val cols: Int,  
    val data: DoubleArray // row-major, flat  
) {  
    init {  
        require(data.size == rows * cols)  
    }  
  
    constructor(nested: List<List<Double>>) : this(  
        nested.size,  
        nested.firstOrNull()?.size ?: 0,  
        DoubleArray(nested.size * (nested.firstOrNull()?.size ?: 0)).also { flat  
->  
            val c = nested.firstOrNull()?.size ?: 0  
            for (i in nested.indices) {  
                val row = nested[i]  
                require(row.size == c)  
                for (j in 0 until c) flat[i * c + j] = row[j]  
            }  
        }  
    )  
}
```

Faster matrices multiplication

$$A = X \cdot W$$

Support for SIMD - Loop reorder and manual hoisting

```
fun fasterMatmul(other: Matrix): Matrix {
    val m = rows val k = cols
    val n = other.cols; val a = this.data
    val b = other.data; val c = DoubleArray(m * n)    // zero-initialized
    var i = 0
    while (i < m) {
        val iC = i * n
        val iA = i * k
        var kk = 0
        while (kk < k) {
            val aik = a[iA + kk]           // hoisted scalar
            val bRow = kk * n              // hoisted base index
            if (aik != 0.0) {               // optional: skip zeros (sparse-friendly)
                var j = 0
                while (j < n) {
                    c[iC + j] += aik * b[bRow + j]
                    j++
                }
            }
            kk++
        }
        i++
    }
}
```


8x Faster matrices multiplication $A = X \cdot W$

Support for SIMD - Loop reorder and manual hoisting

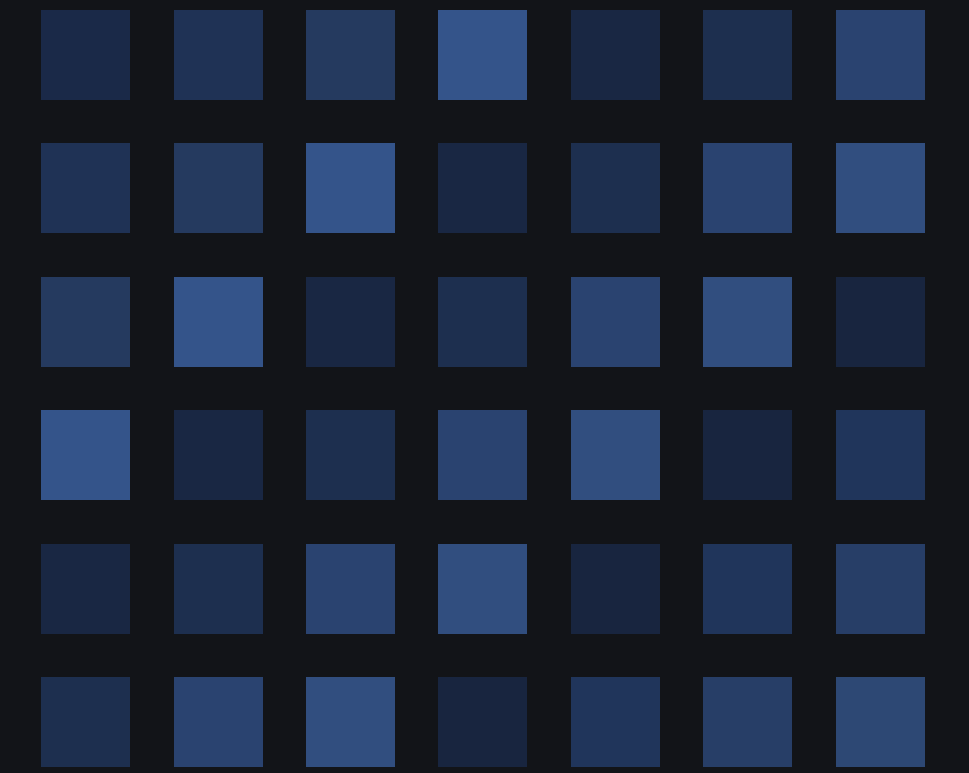
```
fun fasterMatmul(other: Matrix): Matrix {
    val m = rows val k = cols
    val n = other.cols; val a = this.data
    val b = other.data; val c = DoubleArray(m * n)
    var i = 0
    while (i < m) {
        val iC = i * n
        val iA = i * k
        var kk = 0
        while (kk < k) {
            val aik = a[iA + kk] // hoist
            val bRow = kk * n // hoist
            if (aik != 0.0) { // opt
                var j = 0
                while (j < n) {
                    c[iC + j] += aik * b[bRow + j]
                    j++
                }
            }
            kk++
        }
        i++
    }
}
```

**Premature
Optimization
???**

81% of the compute in GTP-2 XL

is general matrix multiplication

$$A = X \cdot W$$



81%



GEMM

19%



Other
operations

Why it matters

Every speedup in matrix kernels
compounds across training and
inference.

ReLU

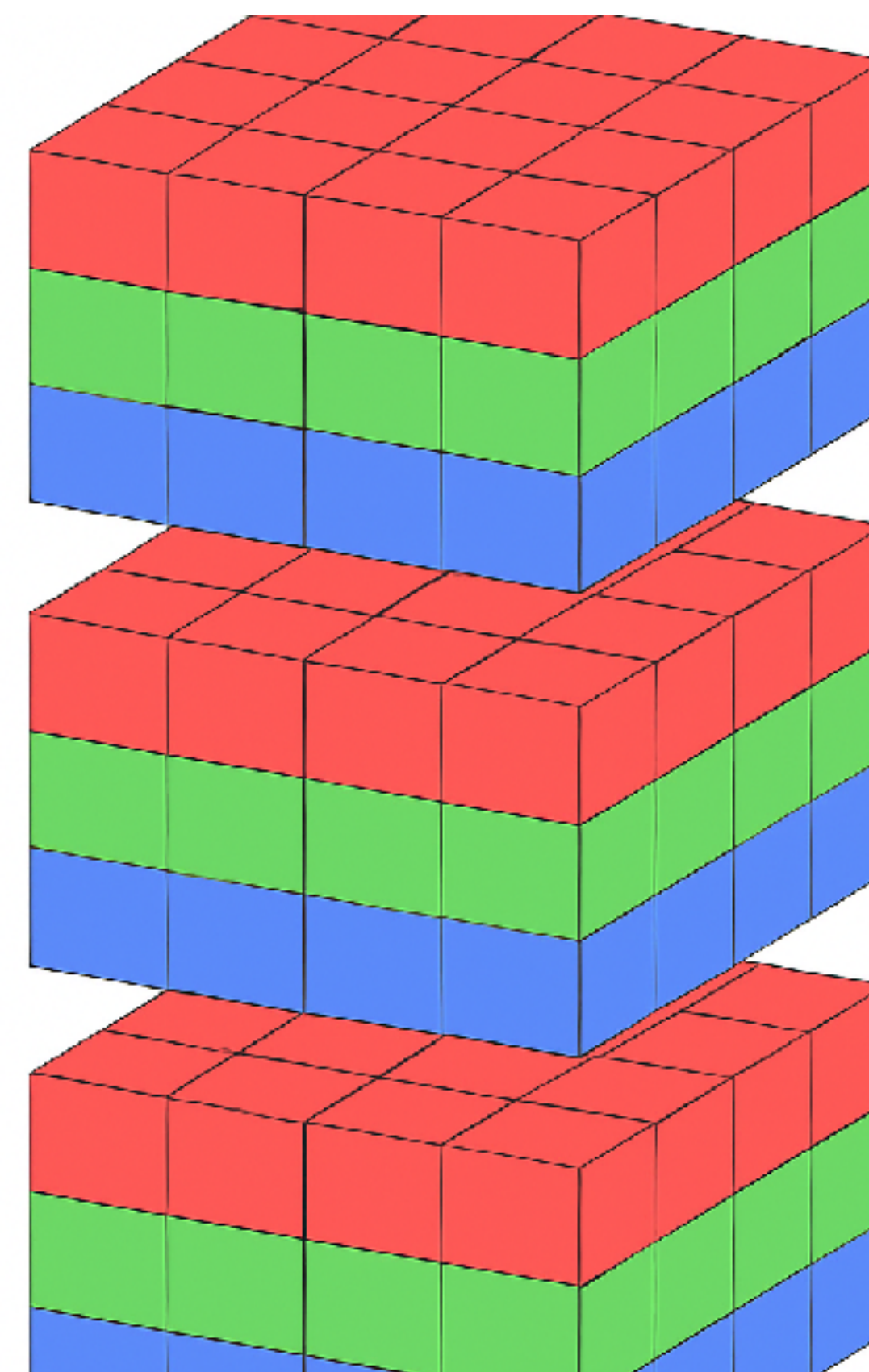
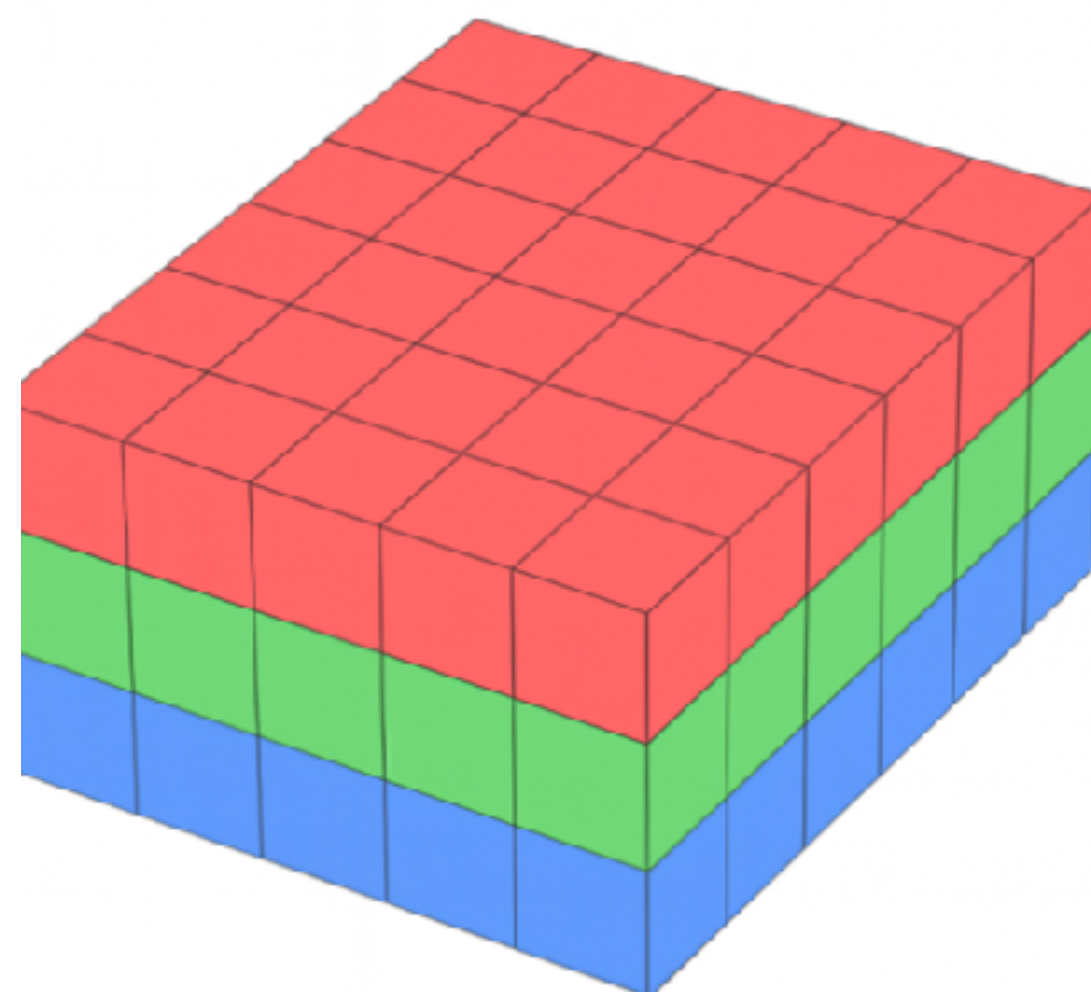
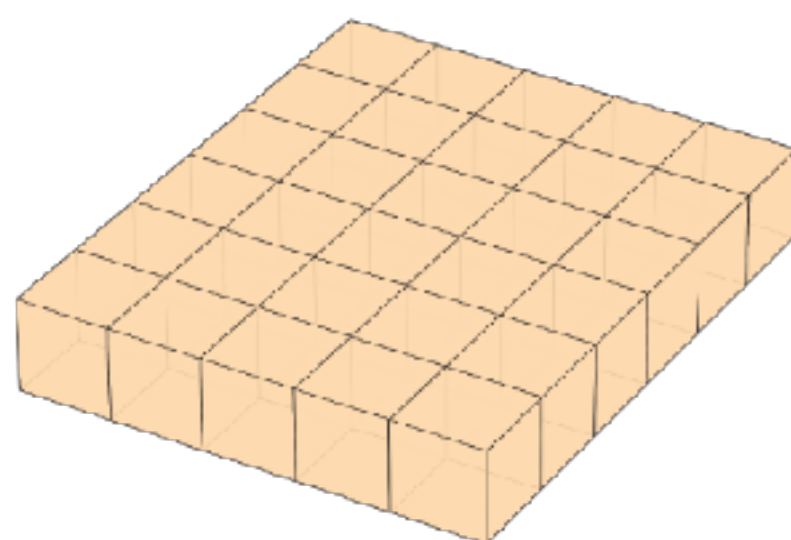
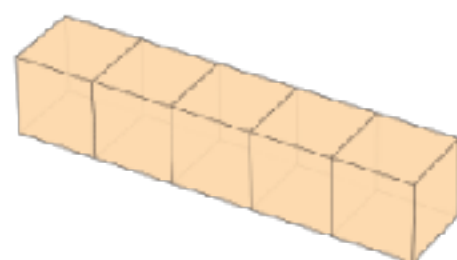
Activation function

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

```
fun Matrix.relu(): Matrix {  
    val src = this.data  
    val dst = DoubleArray(src.size)  
    for (i in src.indices) {  
        dst[i] = if (src[i] > 0.0) src[i] else 0.0  
    }  
    return Matrix(rows, cols, dst)  
}
```

ML/AI Data structures

Scalar



Scalar

Vector

Matrix

Tensor

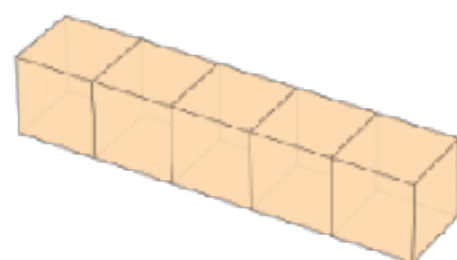
Batch

ML/AI Data structures

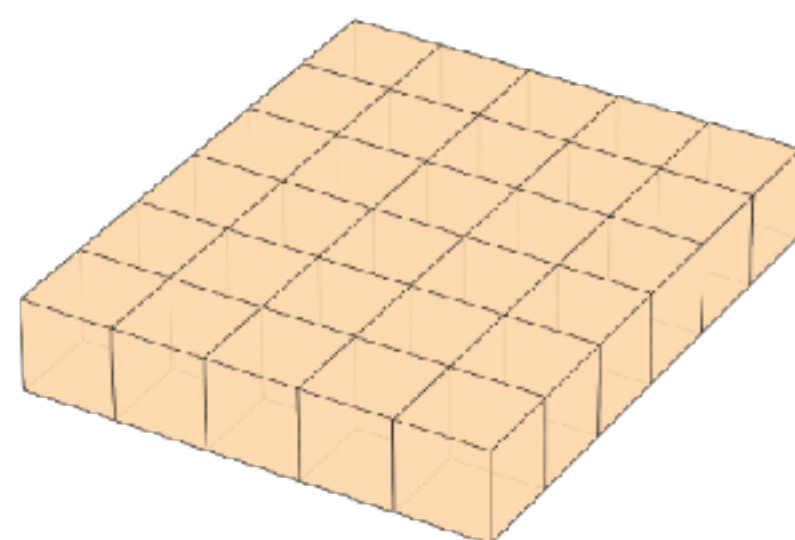
All tensors with a rank



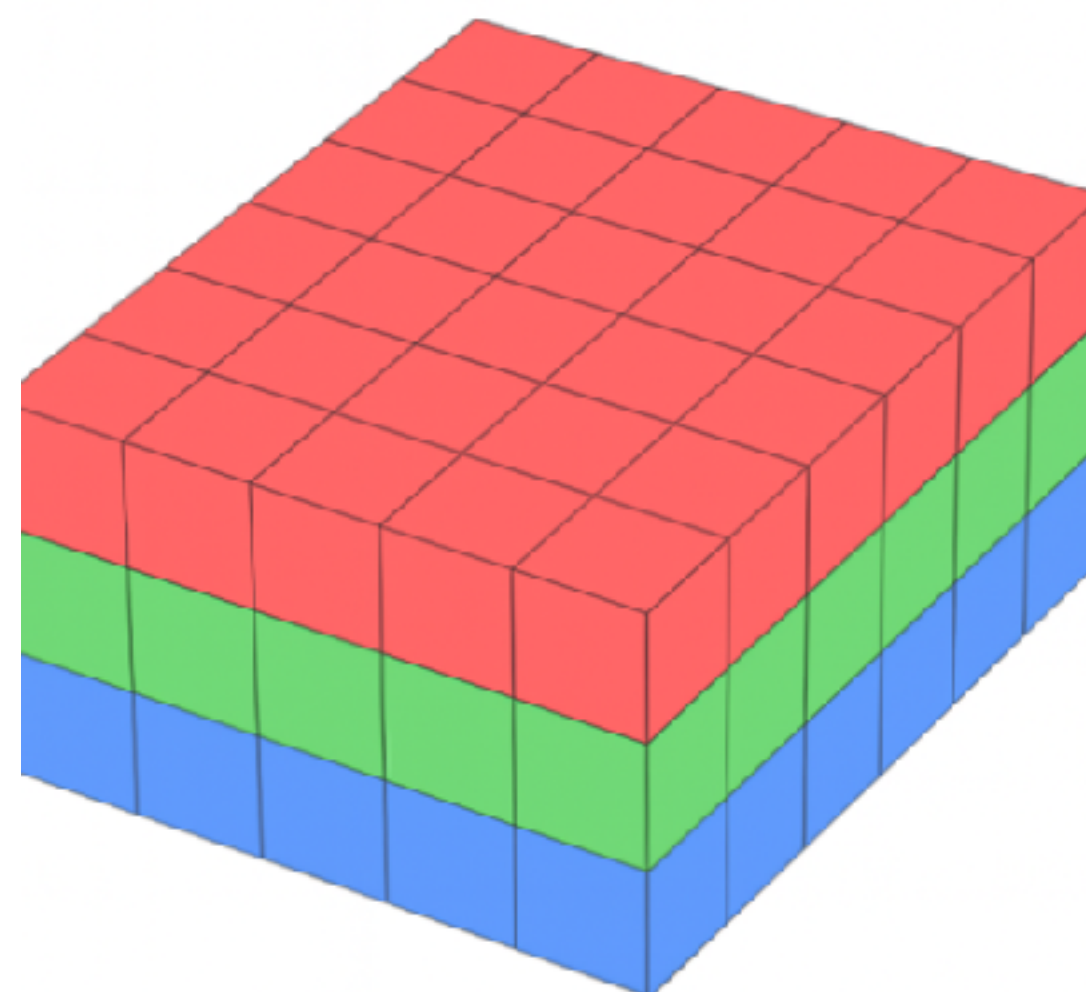
Scalar
Rank=0



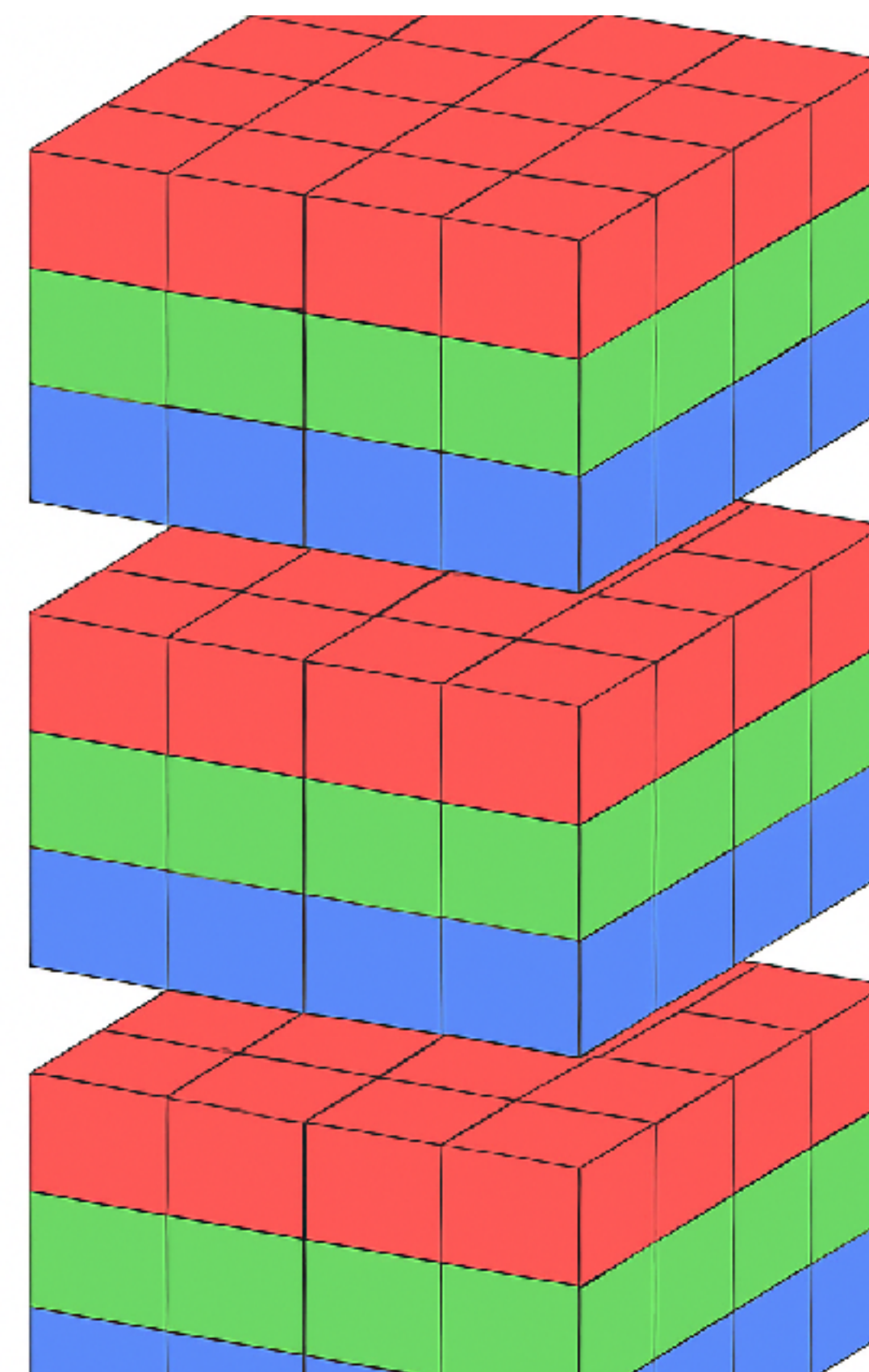
Vector
Rank=1



Matrix
Rank=2



Tensor
Rank=3



Tensor
Rank=4

Memory layout

Tensors

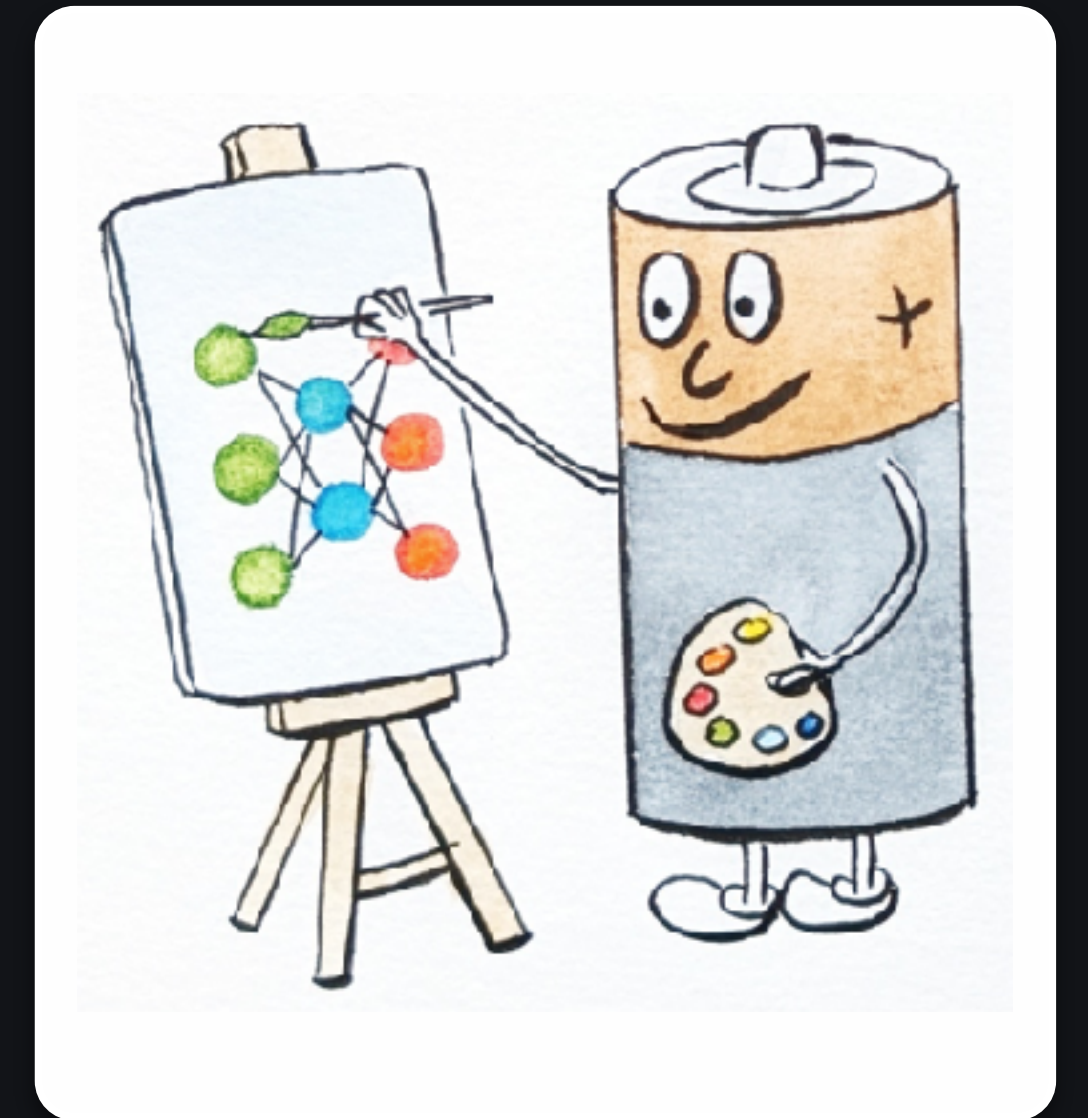
- Elements order - **Row major**
- convention
 - BCWH, BWHC -picture
 - B, T, C - Transformers

1	2	3		7	8	9		13	14	15
4	5	6		10	11	12		16	17	18

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----

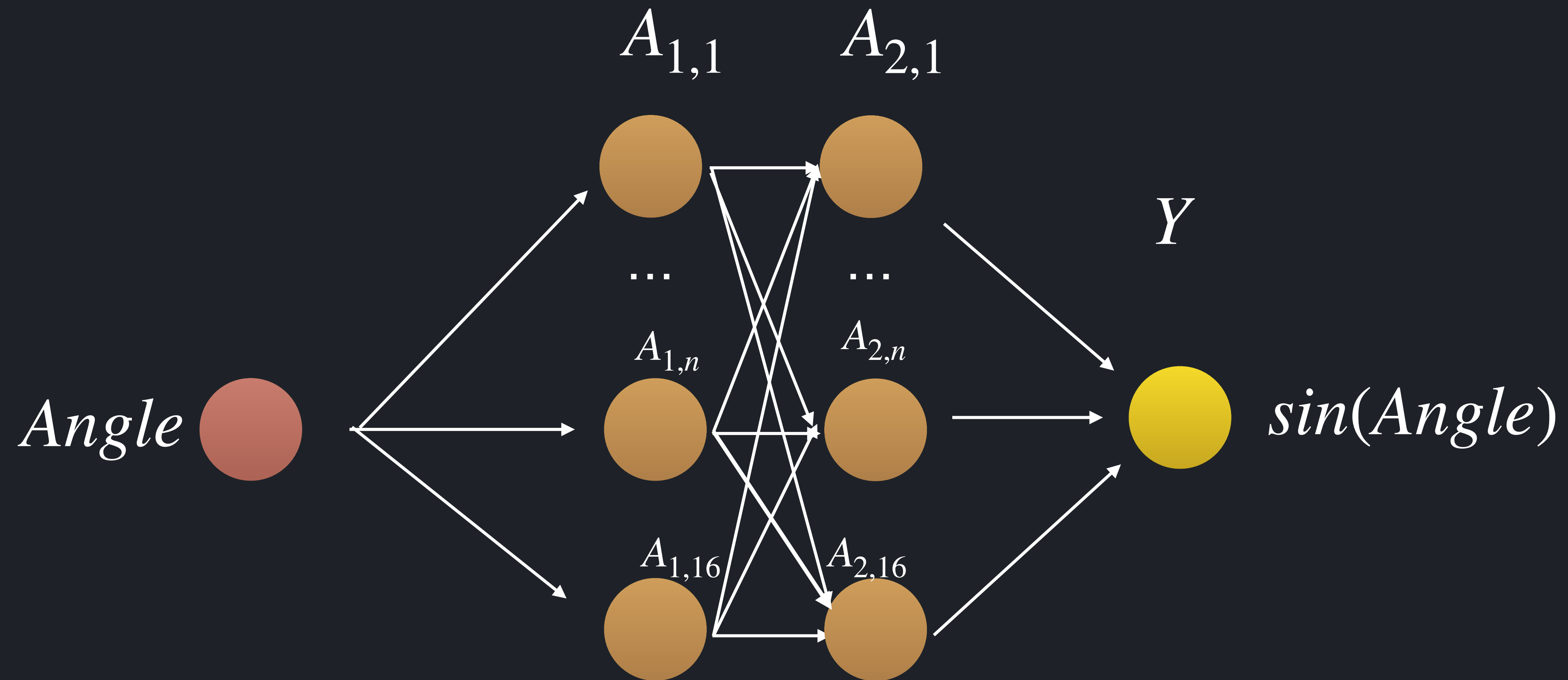
Simple deep neural network example

Feed
Forward



(C) Adriana Harakalova, 2024

Sinus approximation with neuronal network



Forward Propagation in Kotlin

Hidden layer

```
class Linear() {  
    val weights: Tensor  
    val bias: Tensor  
  
    fun forward(input: Tensor): Tensor {  
        val output = input.matmul(weights.value.t()) + bias.value  
        return output  
    }  
}
```


Forward Propagation in Kotlin

MLP

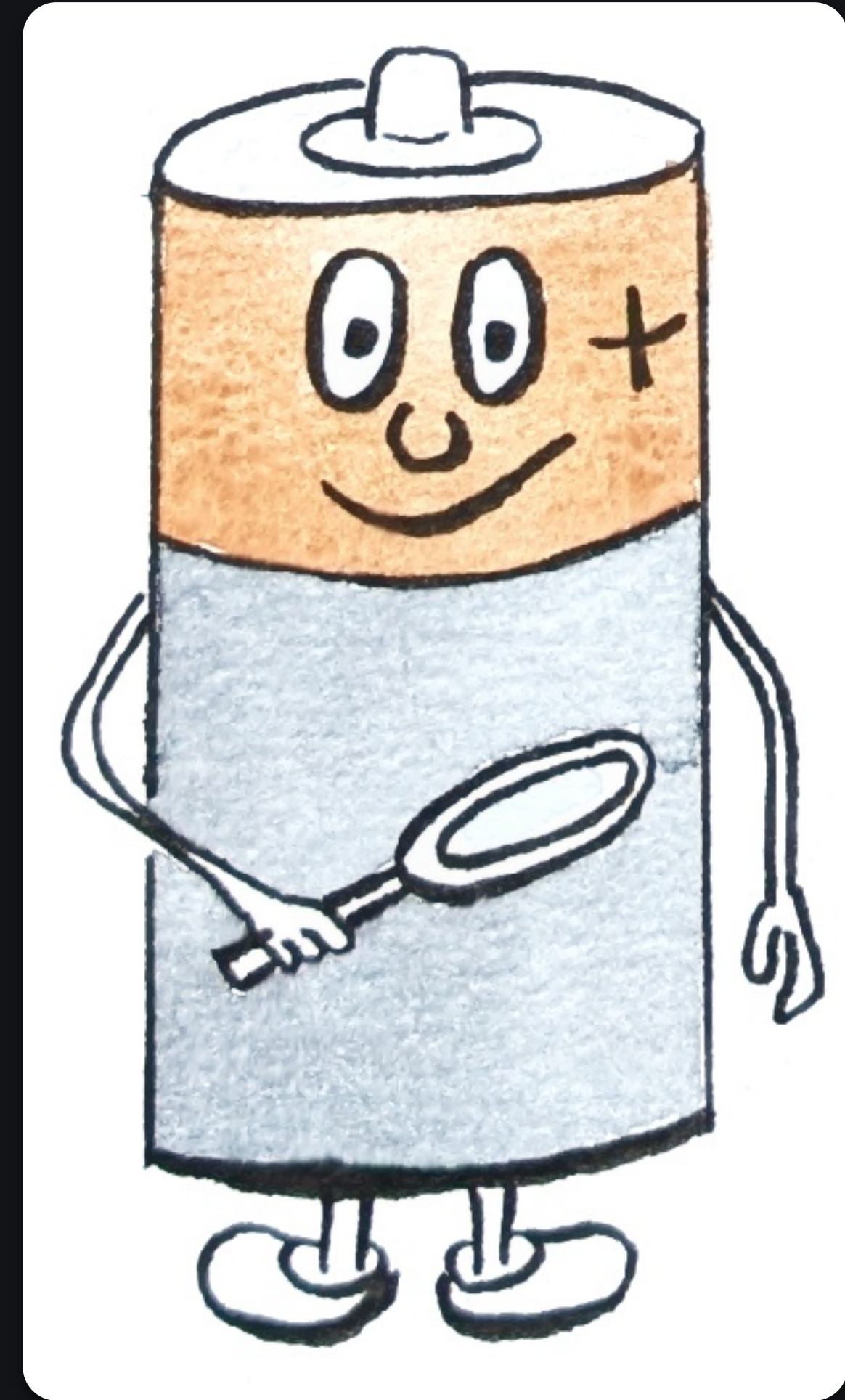
```
class SineMLP : Module() {  
    private val inputLayer: Module = Linear(1, 16)  
    private val hiddenLayer1: Module = Linear(16, 16)  
    private val hiddenLayer2: Module = Linear(16, 16)  
    private val outputLayer: Module = Linear(16, 1)  
    private val relu: Module = ReLU()  
  
    override fun forward(input: Tensor): Tensor {  
        var x = input  
        x = inputLayer.forward(x)  
        x = relu.forward(x)  
        x = hiddenLayer1.forward(x)  
        x = relu.forward(x)  
        x = hiddenLayer2.forward(x)  
        x = relu.forward(x)  
        x = outputLayer.forward(x)  
        return x  
    }  
}
```

Forward Propagation in Kotlin

MLP with DSL

```
class SineNN(override val name: String="sin") : Module() {  
    private val sineModule = sequential{  
        input(1)  
        linear(16, "layer1") {  
            activation = relu  
        }  
        linear(16, "layer2") {  
            activation = relu  
        }  
        linear(1, "output_layer")  
    }  
    override fun forward(input: Tensor): Tensor =  
        sineModule.forward(input)  
}
```


Reality check



ML Babylon

Many languages, many platforms, many technologies

Languages

- **Halide**
 - Focus on Pictures and Tensors
- **Julia**
 - Dynamically types
 - Flux.jl Lux.jl for **ML**
 - popular in academia

HW/Plattformen

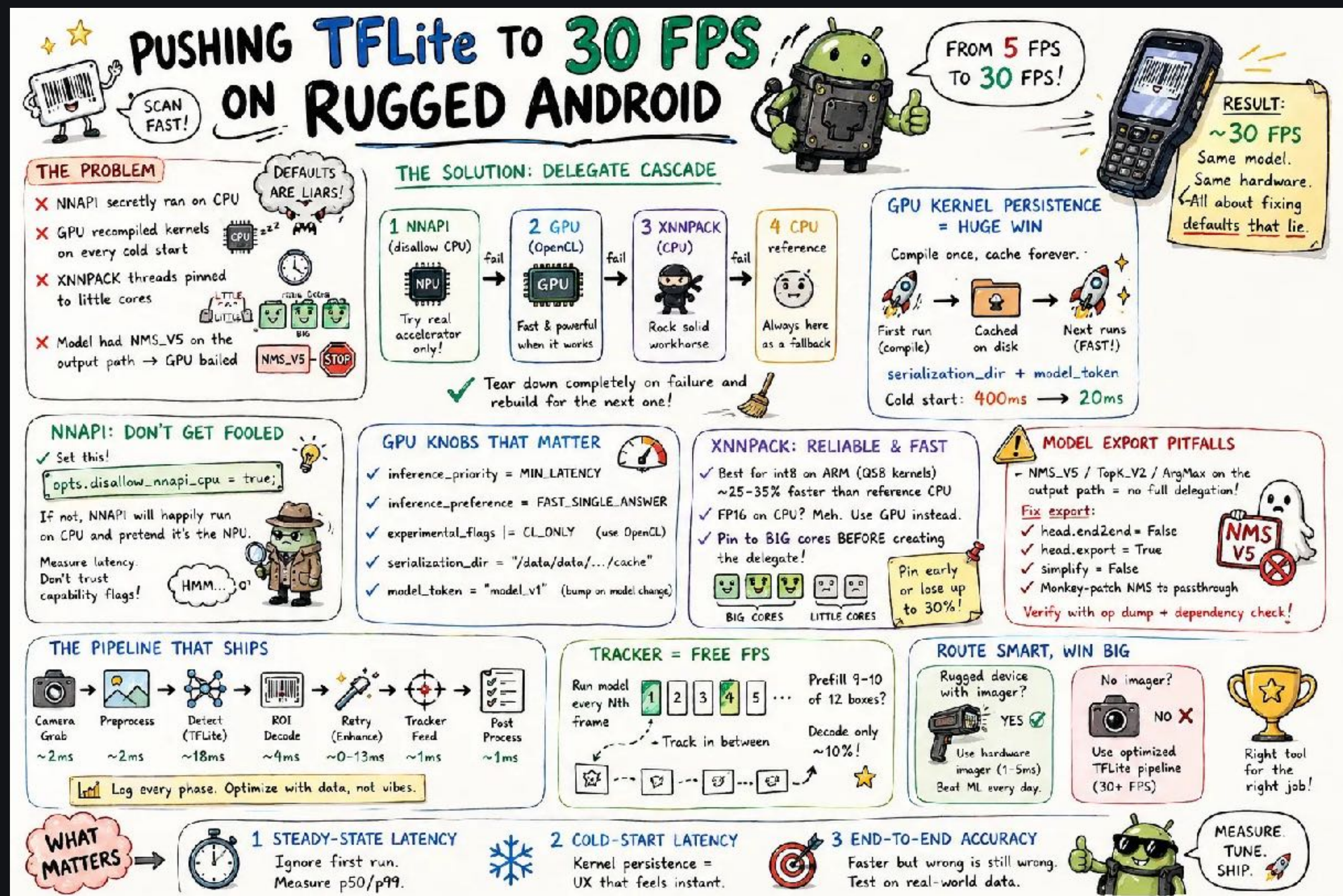
- **CUDA**
 - **Nvidia**
- **Metal**
 - Apple
- **Vulcan, NPU**
 - Android

Focus on AI

- Mojo
- Pythonic
- MLIR / LLVM IR
- ONNX

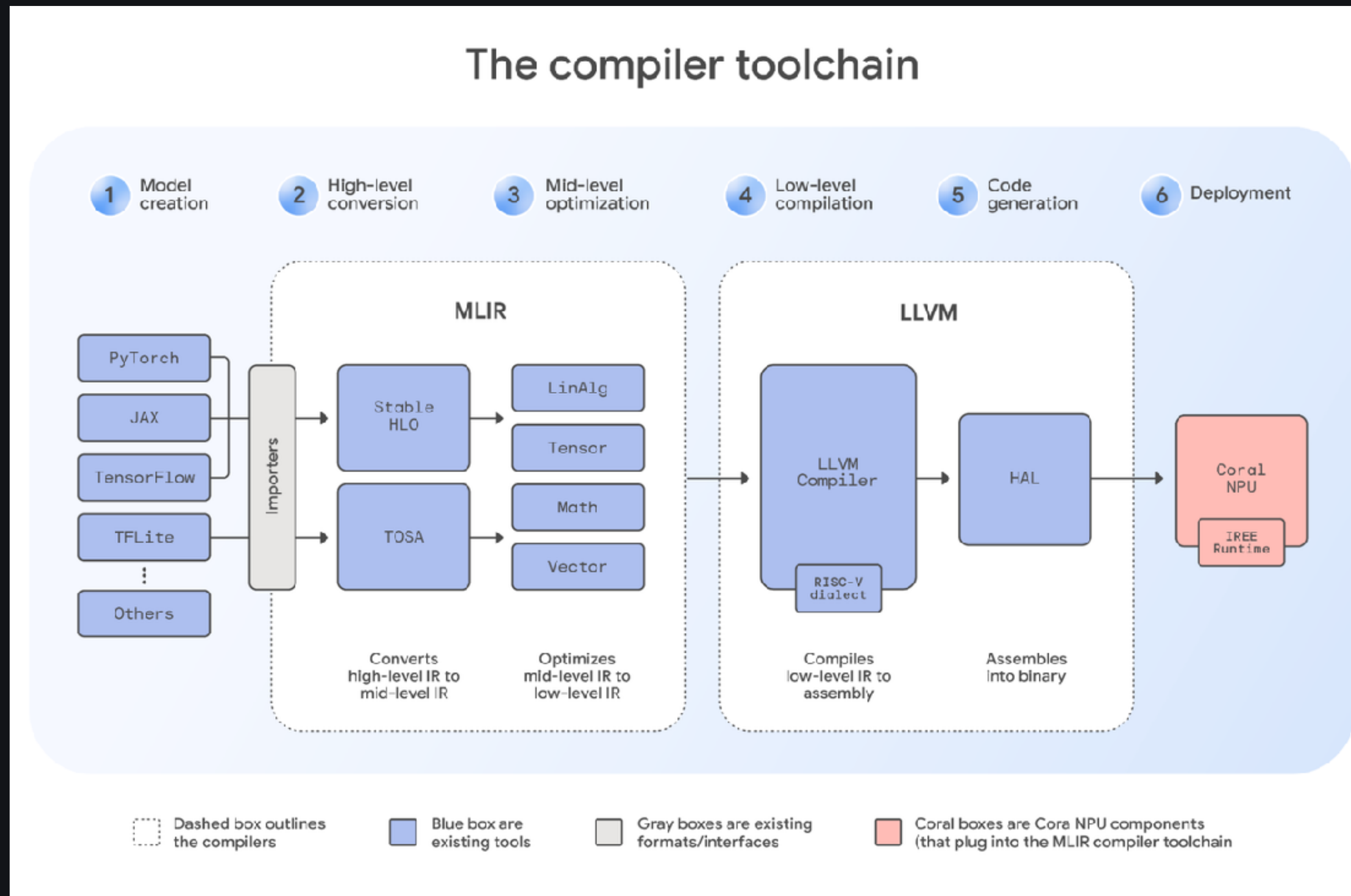
From 5 fps to 30 fps: What the TFLite Defaults Are Hiding

Kashif Mehmood



Google - Coral NPU

Compiler based approach, November 2025



Let's start with a language

One language to find them

One language to bind them

Modern

One language to find them

One language to bind them



Typesafe

One language to find them

One language to bind them

Support for concurrent/
asynchronous programming

One language to find them

One language to bind them

Multiplatform

One language to find them

One language to bind them

Modern

One language to find them

One language to bind them



Typesafe

One language to find them

One language to bind them

Support for concurrent/
asynchronous programming

One language to find them

One language to bind them

Multiplatform

One language to find them

One language to bind them

Modern

One language to find them

One language to bind them



Typesafe

One language to find them

One language to bind them

Coroutines

One language to find them

One language to bind them

Multiplatform



Kotlin is the Language for AI

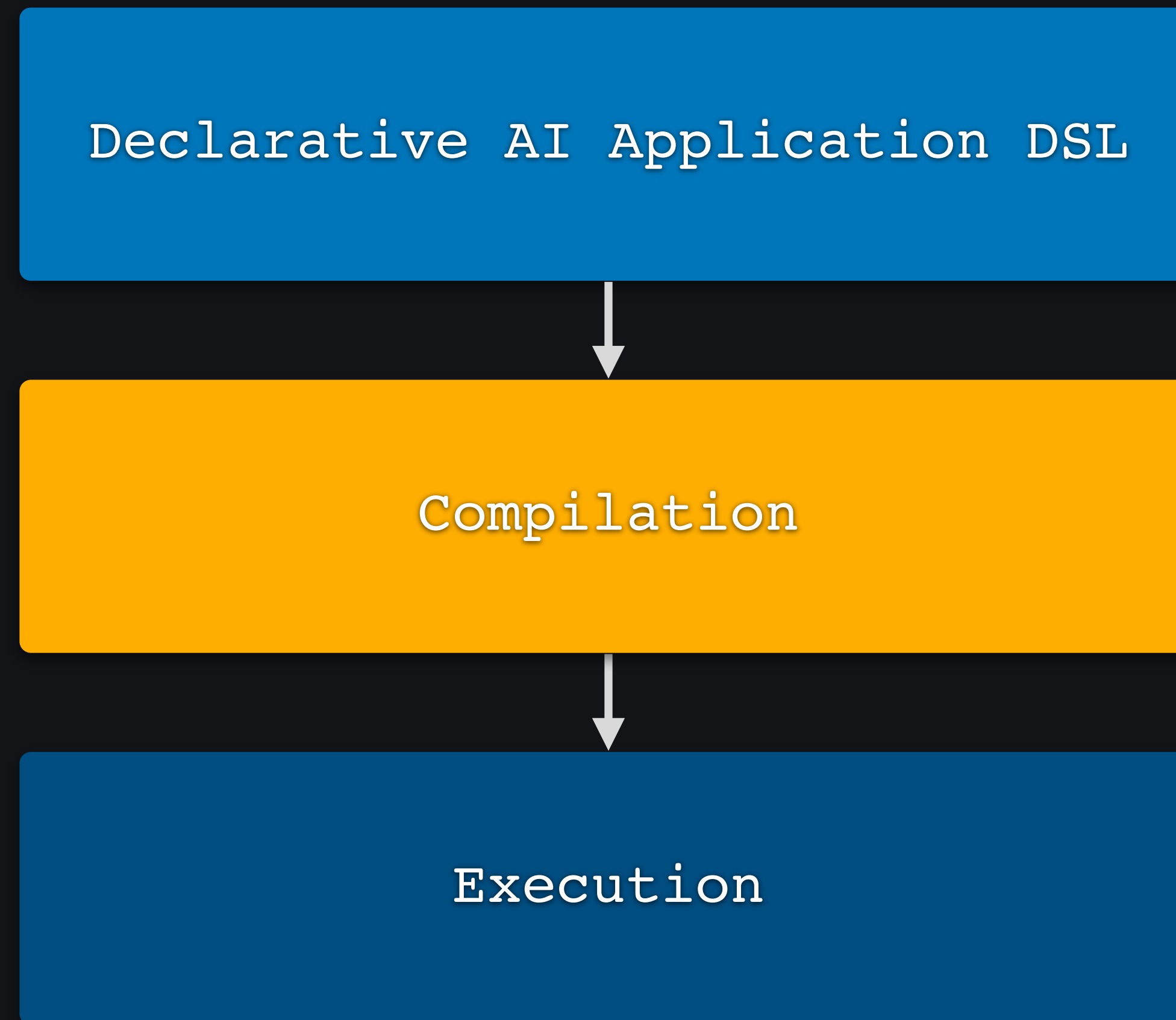
Kotlin is the Language for AI

Kotlin

- Supports DSLs for readable, domain-specific code
- Type-safe and less error-prone
- Concise, readable syntax
- Supports coroutines for asynchronous/concurrent programming
- Offers compiler plugins and tooling
- Multiplatform

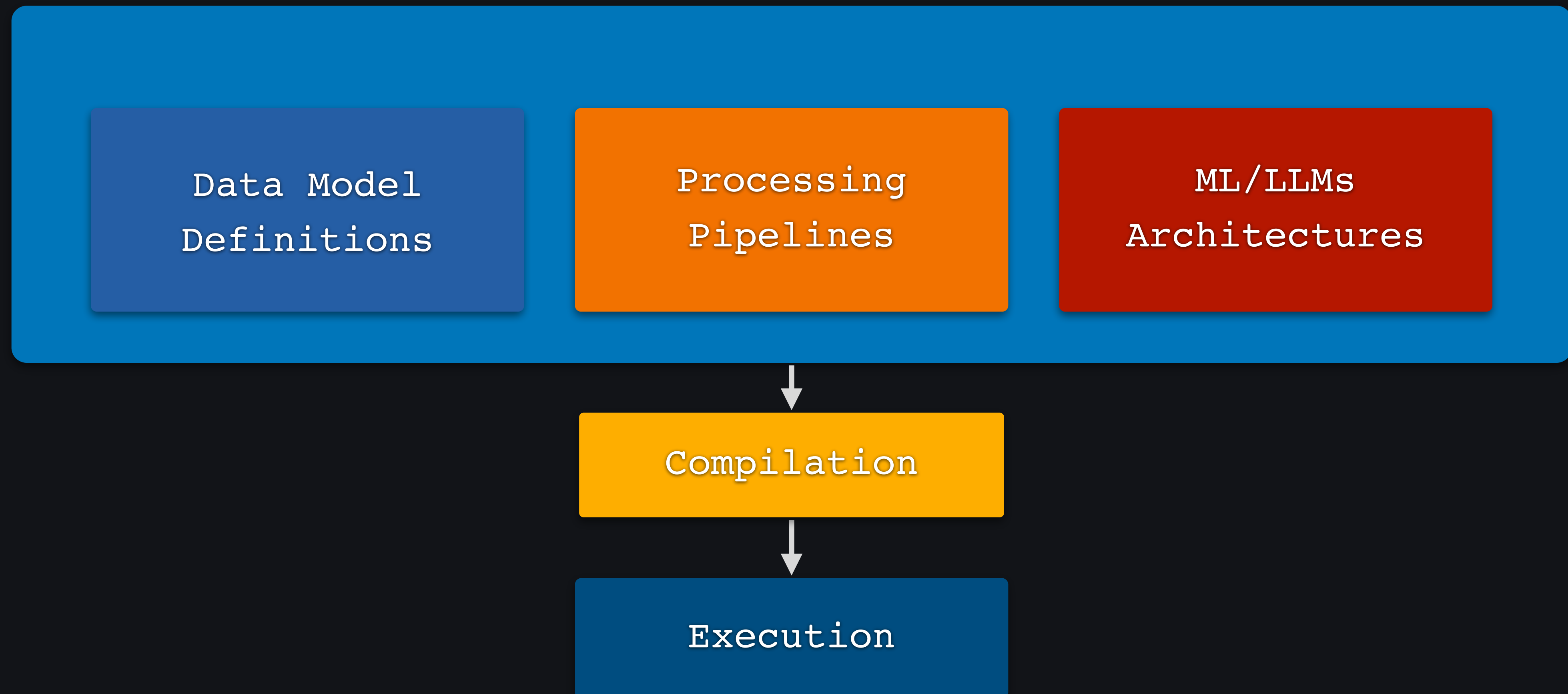
Build AI from scratch in Kotlin

Redefining on-device AI with Kotlin



Kotlin DSL

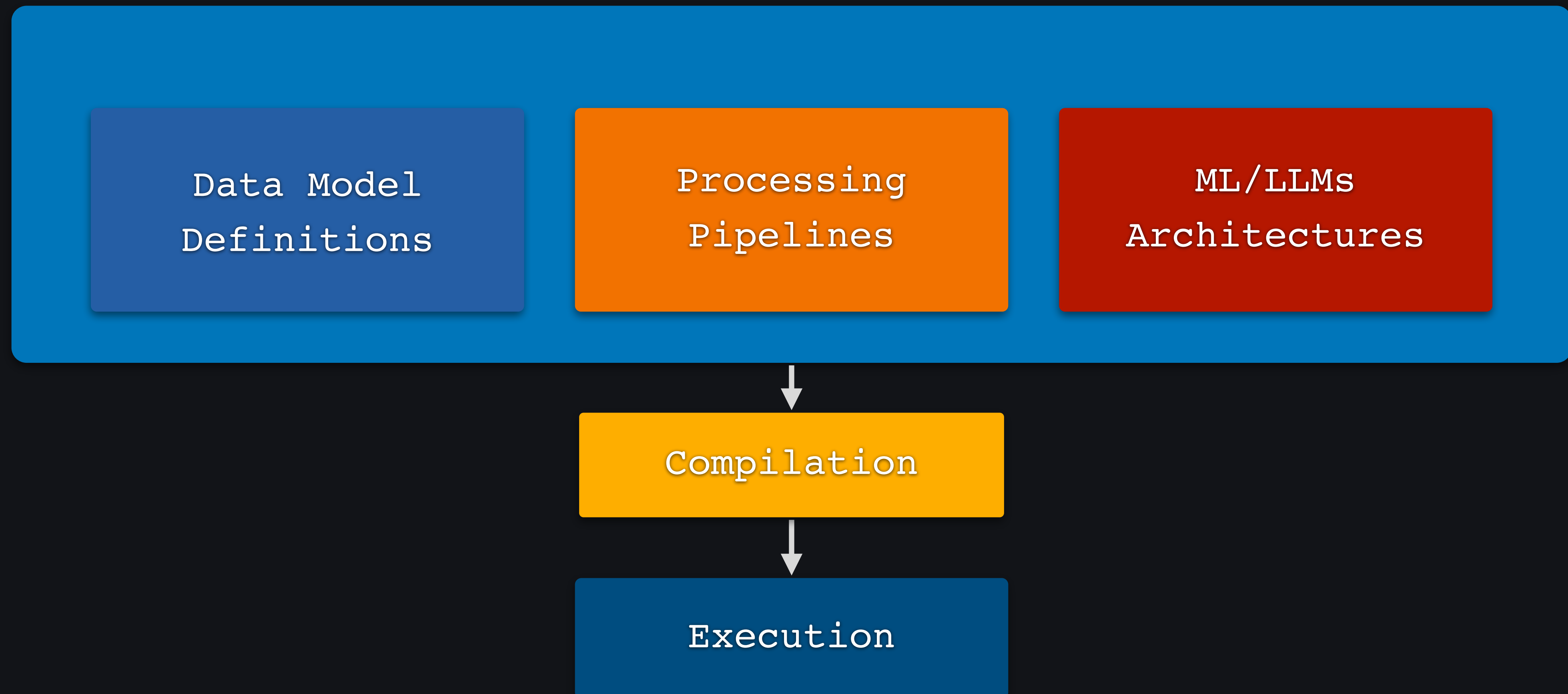
Turning an AI application into a structured Kotlin definition.



Data Model Definitions

```
data {  
    // RGB image  
    val input = tensor<FP32, Float> {  
        shape(640, 480, 3) {  
            zeros()  
        }  
    }  
  
    // matrix  
    val mask = tensor<INT8, Int> {  
        shape(10, 10) {  
            zeros()  
        }  
    }  
}
```

Kotlin DSL

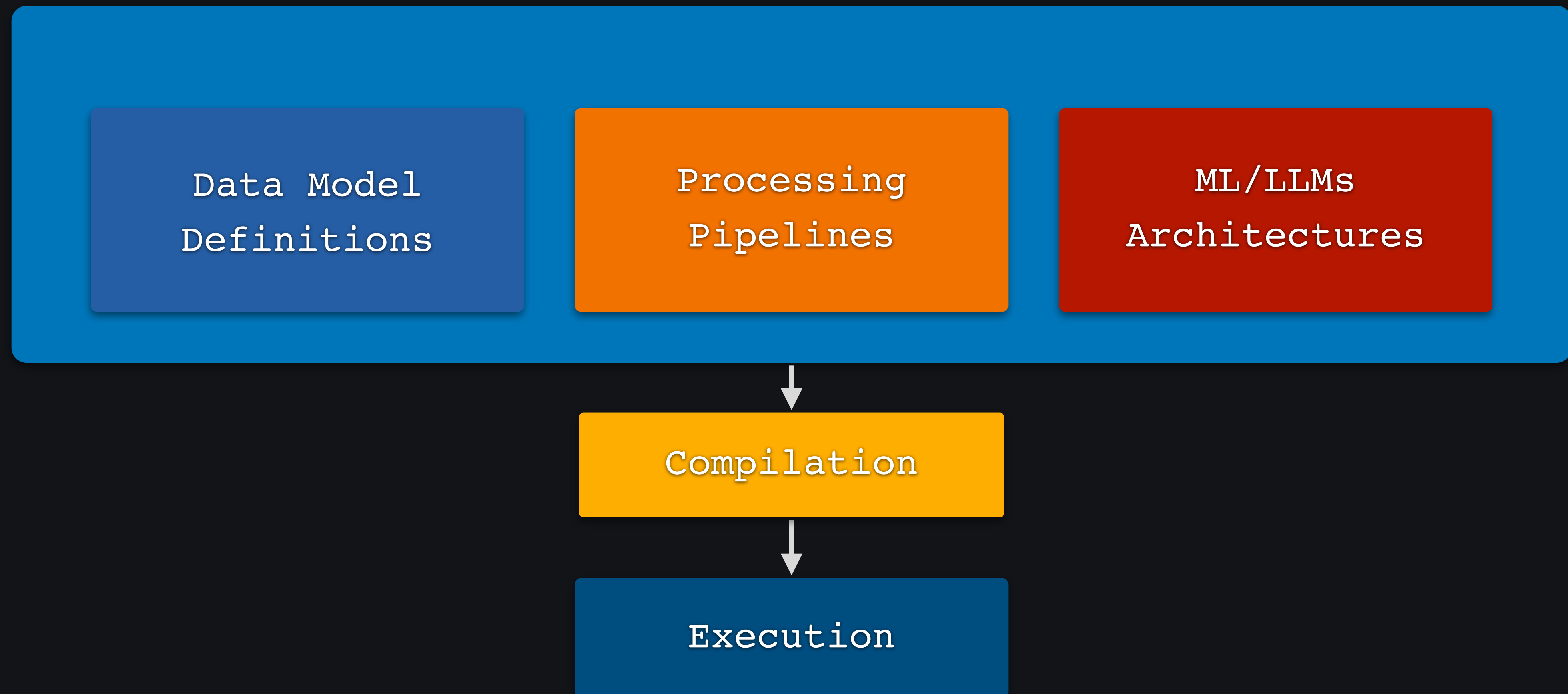


Compilation

- Kotlin KSP
- LLVM.org compiler tools
 - MLIR
- IREE.org



Compilation



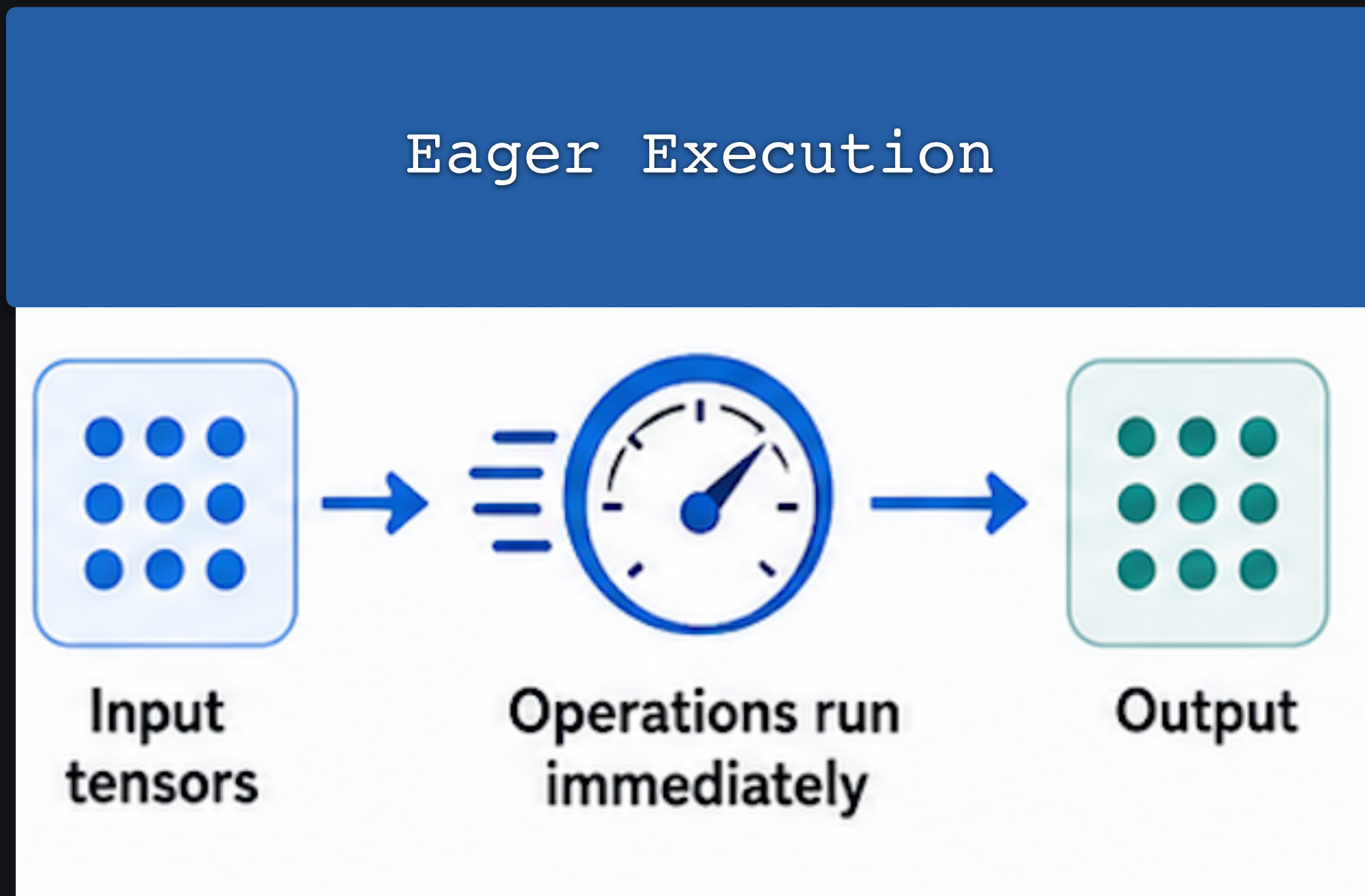
Eager
Execution

Graphmode

Code generation

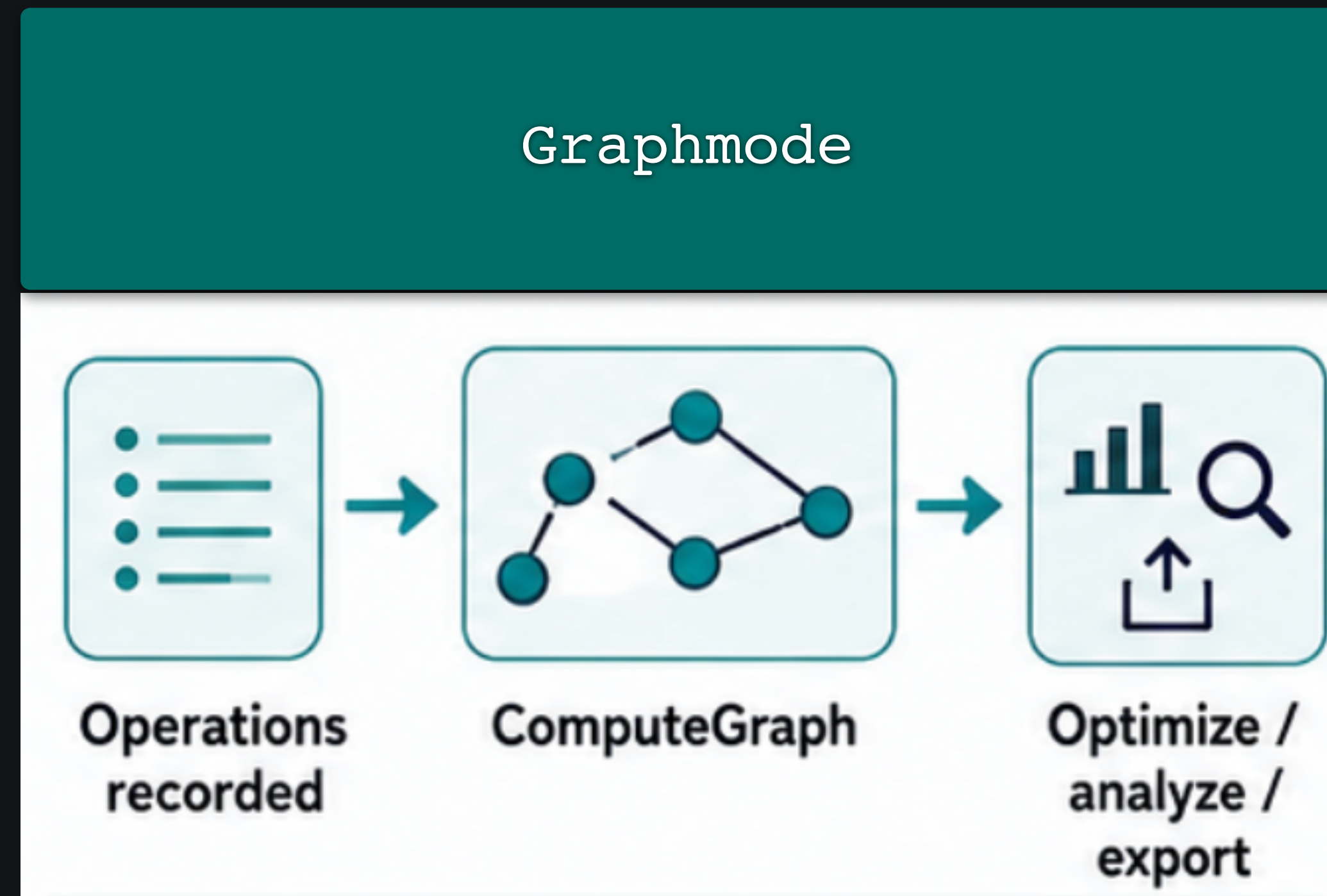
Eager Execution

DirectCpuExecutionContext



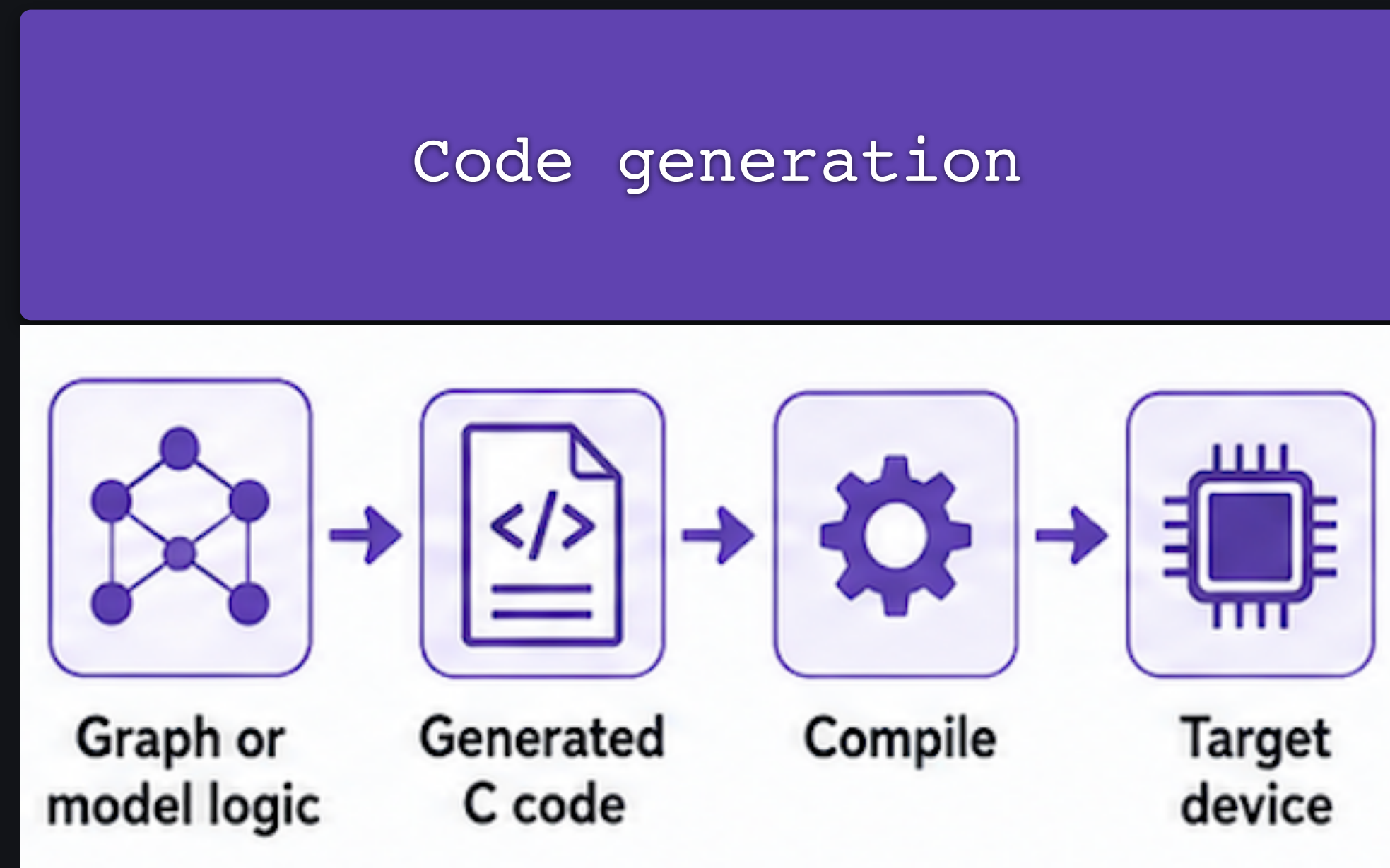
Graph mode

DefaultGraphExecutionContext



Code generation

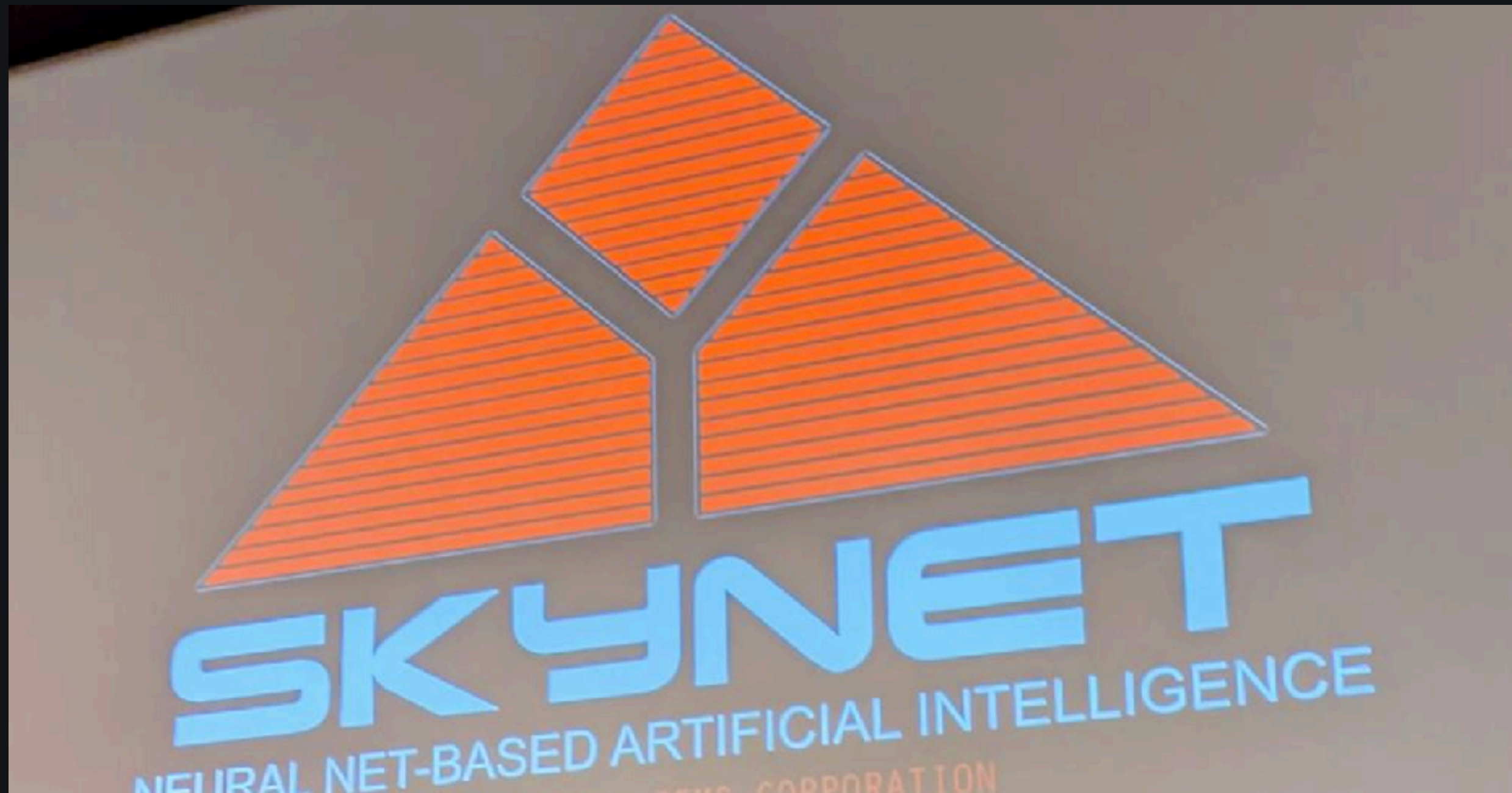
Compiled deployment path



One more thing ...

SKaiNET

Naming is hard - a working title

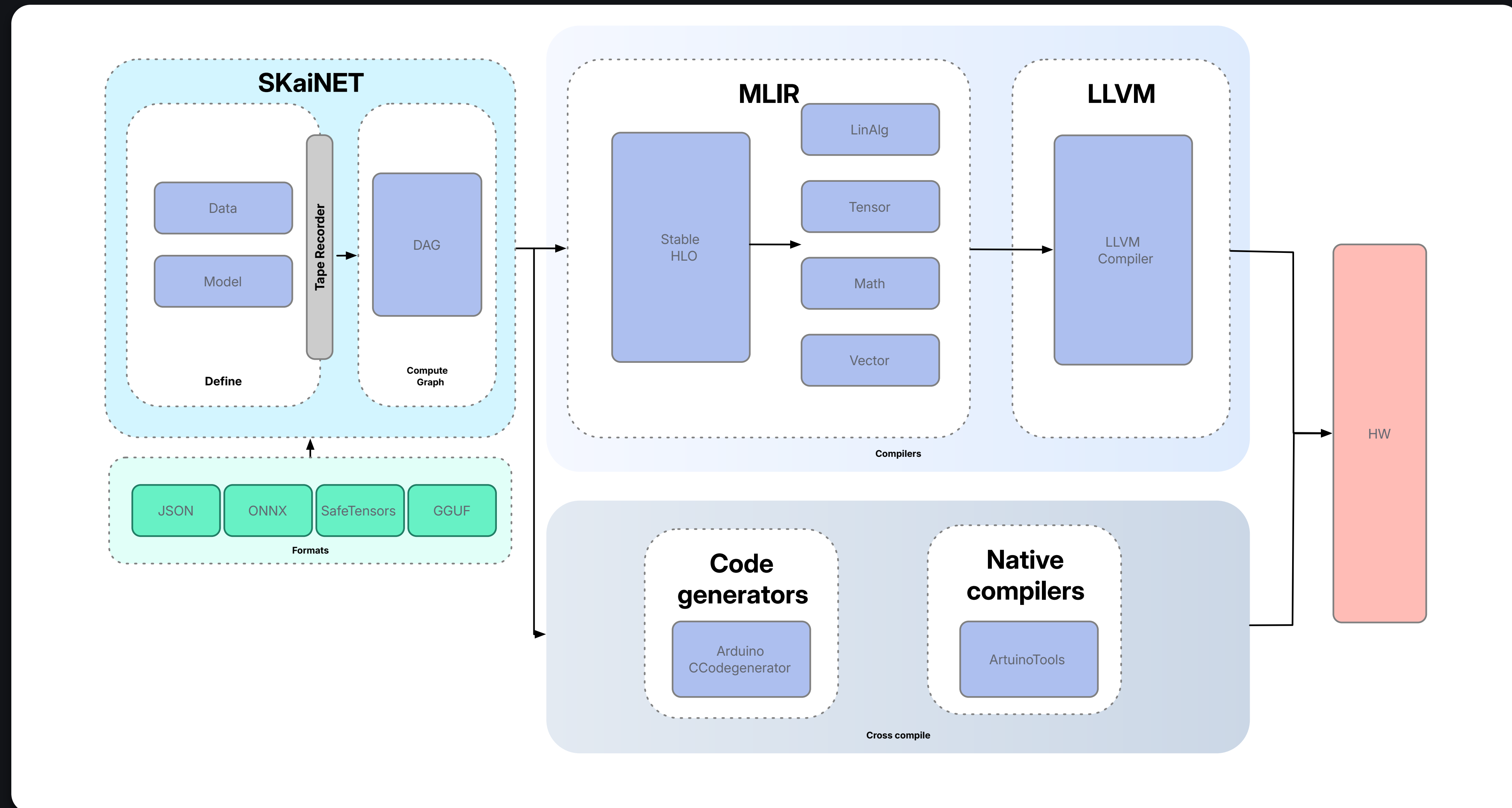


Yet Another ML/AI Framework ?

- On-device-first
- Platform- and hardware-agnostic
- Developer-focused
- Kotlin powered

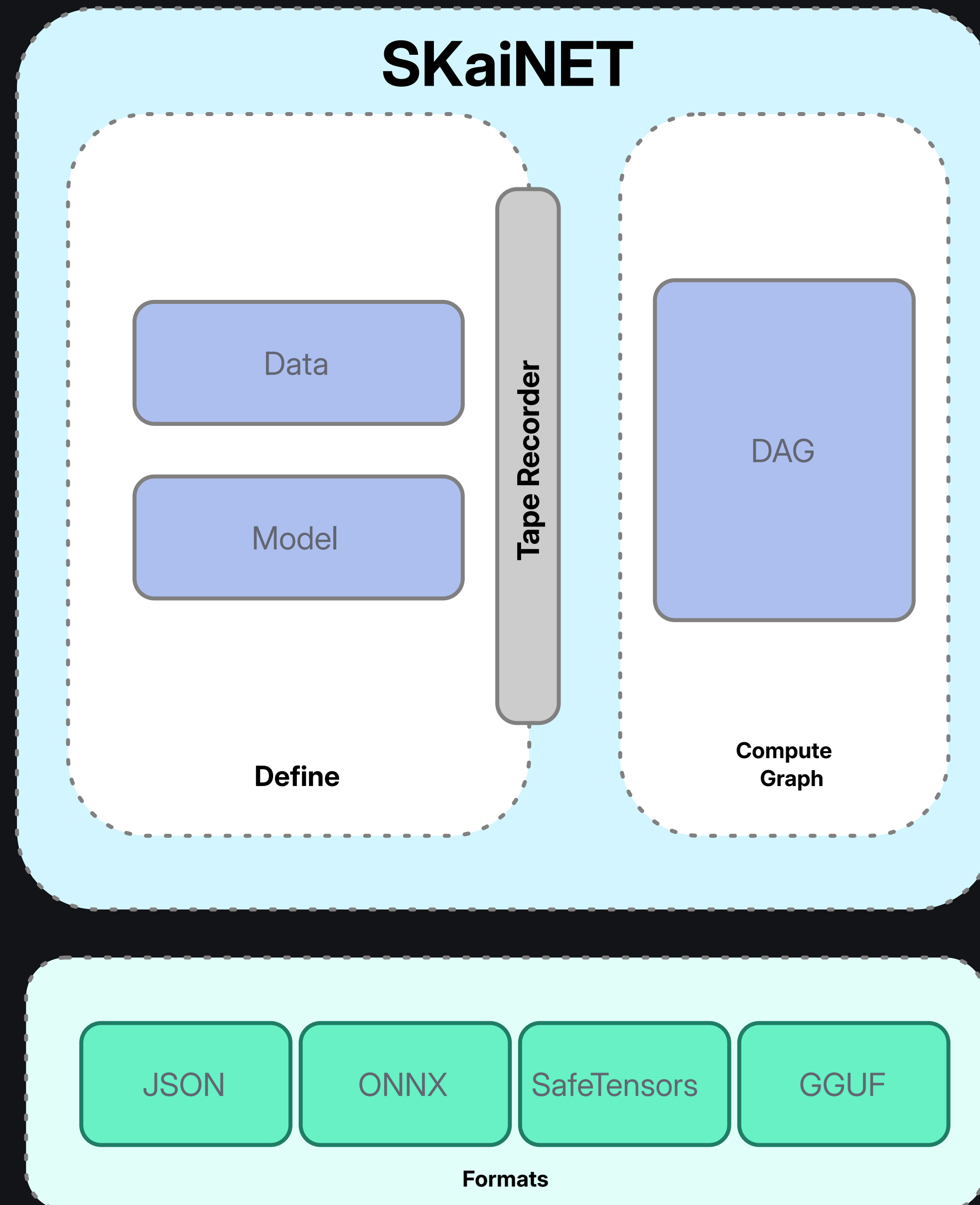
Architecture

Standing on the shoulders of giants



SKaiNET

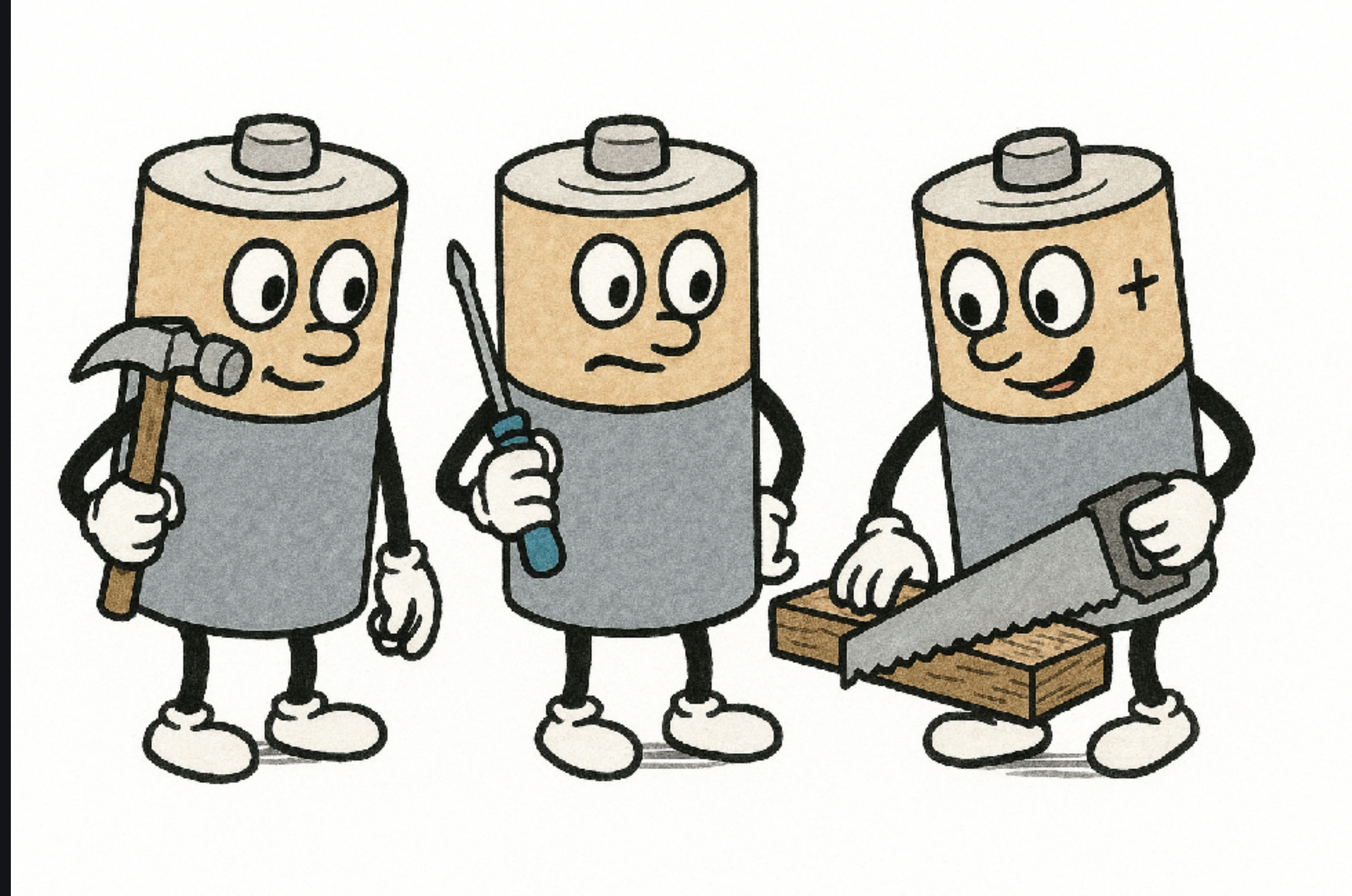
Core



SKaiNET is open source

MIT license

- Collaboration
- OSS compliance effort



Open Source project development

Bringing data scientist and developer together

- I can Kotlin
- I can machine learning

DARC

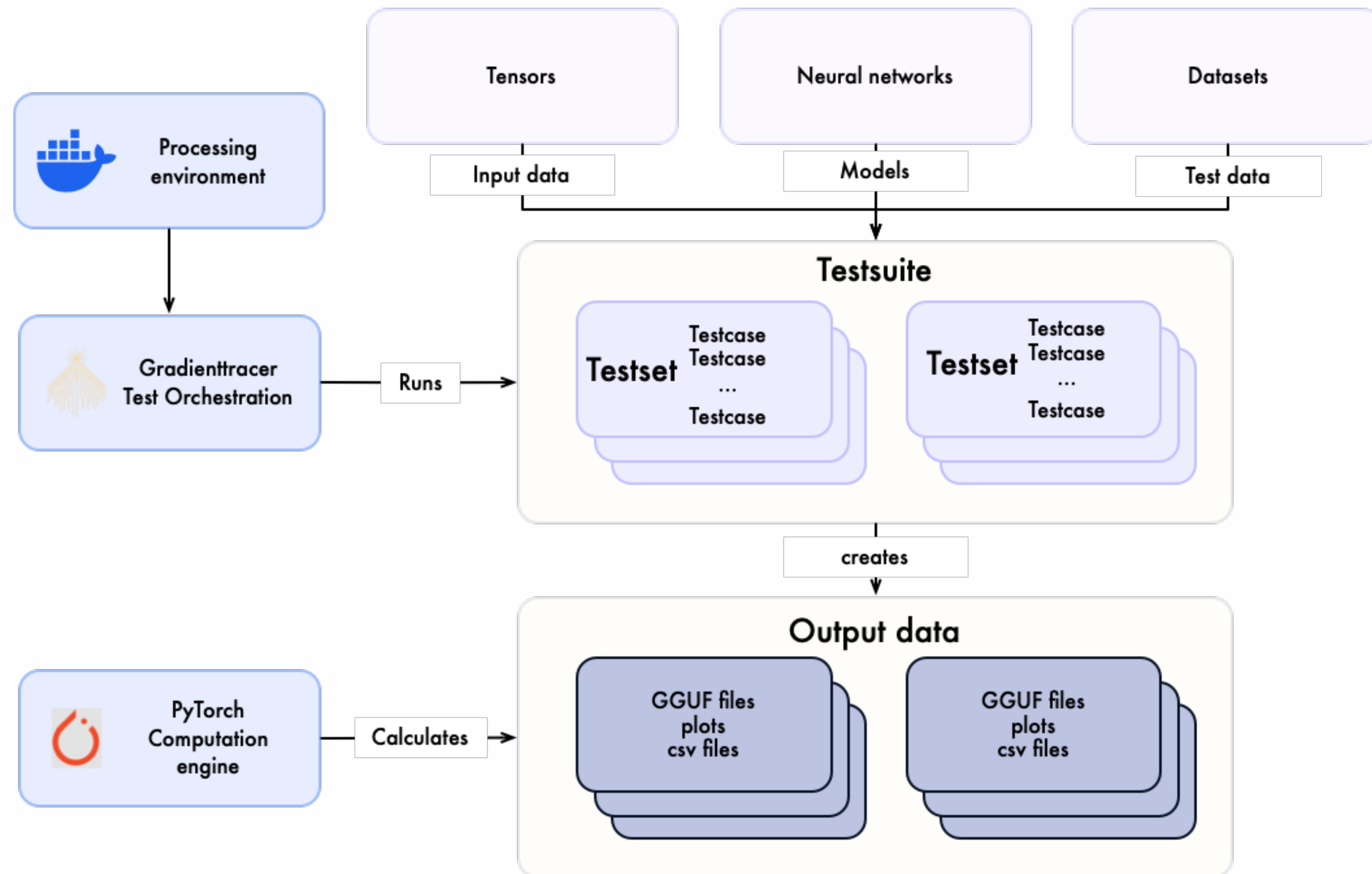
Collaboration workflow

- Document
- Assess
- Research
- Code



Validation

Create Ground Truth



Demo



<https://harakal.de>

Thank You, and Don't Forget to Vote

